Intelligent Warehouse Recommendation for Maharashtra Using Hybrid Machine Learning and LLM Assistance

Submitted in partial fulfilment of the requirements of the degree of

BACHELOR OF COMPUTER ENGINEERING

by

Dipesh Sharma, 22102035

Manthan Shinde, 22102124

Sahil Shinde, 22102013

Pranjal Zambre, 22102118

Guide:

Prof. Varsha Wangikar

Department of Computer Engineering

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "**Intelligent Warehouse Recommendation for Maharashtra Using Hybrid Machine Learning and LLM Assistance**" is a bonafide work of **"Dipesh Sharma" (22102035), "Manthan Shinde" (22102124)**, **"Sahil Shinde" (22102013), "Pranjal Zambre" (22102118)**, submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Engineering**

Prof. Varsha Wangikar
Guide

Dr. Rushikesh Nikam
Project Coordinator

Dr. Sachin Malave
Head of Department

Dr. Uttam Kolekar
Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for B.E.

This project report entitled ***Intelligent Warehouse Recommendation for Maharashtra Using Hybrid Machine Learning and LLM Assistance*** by ***(Dipesh Sharma, Manthan Shinde, Sahil Shinde, Pranjal Zambre)*** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Engineering***, ***2025-26***.

Examiner Name	Signature
1._________________	______________
2.__________________	_______________

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Dipesh Sharma - 22102035

Manthan Shinde - 22102124

Sahil Shinde - 22102013

Pranjal Zambre – 22102118

Date:

Abstract

The modern logistics and warehousing sector face critical inefficiencies due to fragmented property discovery, static listing platforms, and a lack of intelligent, explainable matching systems. Existing solutions either rely on traditional machine learning models that act as unexplainable "black boxes," or they struggle to process unstructured natural language requirements and complex, non-linear pricing models. Furthermore, manual regulatory verification creates significant bottlenecks for onboarding new properties. To address these challenges, this project introduces SmartSpace: a centralized, artificially intelligent warehouse rental marketplace designed to streamline the end-to-end lifecycle of property discovery, inventory management, and automated compliance.

SmartSpace achieves this through a highly resilient, dual-engine AI architecture. The system integrates a 5-algorithm machine learning ensemble (incorporating K-Nearest Neighbors, Random Forest, and Gradient Boosting) to deliver mathematically precise property matches based on a comprehensive dataset of over 10,000 augmented records from the Warehousing Development and Regulatory Authority (WDRA) in Maharashtra. Concurrently, it employs the Meta Llama 3.3 70B large language model via the Groq inference engine to process unstructured user queries, extract intent, and provide clear, contextual explanations for its recommendations effectively solving the AI "black box" problem. The platform is divided into three distinct modules: a Seeker Module featuring an NLP-powered Smart Booking Assistant, an Owner Module equipped with dynamic pricing inference, and an Admin Module that utilizes a Convolutional Neural Network (CNN) for the automated verification of compliance documents. Supported by a secure React-Node-Supabase stack and a failover mechanism to ensure uninterrupted service, SmartSpace provides a scalable, multimodal solution that optimizes space utilization and modernizes the on-demand logistics ecosystem.

Keywords: logistics, AI, end-to-end lifecycle, inventory management, automated compliance, machine learning ensemble, large language model, NLP, CNN, multimodal, on-demand logistics

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF SCREENSHOTS

ABBREVATION

CNN	*Convolutional Neural Network*
MIDC	*Maharashtra Industrial Development Corporation*
FMCG	*Fast-moving consumer goods*
SME	*Small and Medium Enterprises*
CCTV	*Closed-Circuit Television*
GST	*Goods and Services Tax*
PAN	*Permanent Account Number*
LLM	*Large Language Model*
GPU	*Graphics Processing Unit*
NLP	*Natural Language Processing*
AI	*Artificial intelligence*
API	*Application programming interface*
WDRA	*Warehousing Development and Regulatory Authority*
JWT	*JSON Web Token*
RBAC	*Role-Based Access Control*
RLS	*Row-Level Security*
SQL	*Structured Query Language*
DFD	*Data Flow Diagram*
VPS	*Virtual Private Servers*

CHAPTER 1

Introduction

The warehousing sector in Maharashtra has experienced strong growth, primarily driven by a high need for storage by small businesses. This rapid expansion creates a pressing need for a smart warehousing system. Currently, there are warehouses spread all over Maharashtra, ranging from large facilities in MIDC areas and conventional godowns to smaller galas and dark stores utilized by quick-commerce companies like Swiggy Instamart and Zepto for fast delivery. These facilities cater to various specialized needs, including cold storage, pharma grade, electronics, FMCG, and general storage varieties. The state's logistics infrastructure demands a structured approach, given that thousands of unverified listings exist, creating a highly fragmented ecosystem.

However, despite this industry growth, the current process of searching for suitable warehouses is still largely manual. It heavily involves the use of brokers and physical site visits, which leads to longer search times, poor decision-making regarding facility selection, and higher operational costs, especially for Small and Medium Enterprises (SMEs). Finding a warehouse is difficult because there are no centralized platforms to view all available options in one place. The problem of warehouse selection is more than just a matter of availability. Decision-makers must simultaneously evaluate multiple factors, such as location, cost limitations, capacity, specialization, compliance, and quality. Furthermore, new or less frequently used warehouses suffer from a "cold-start" problem, remaining practically invisible to potential renters due to a lack of historical booking data, which traditional collaborative filtering techniques fail to overcome.

Current online solutions only address disconnected parts of this problem and lack the capability to make recommendations across multiple criteria with sound reasoning. The normal systems used to help people find things in warehouses are not very effective because they only search for words and use filters that do not change, which means they do not really understand what the user is looking for and they also have problems with questions that are not written in a certain way. The traditional process takes days to call up brokers, visit nearby warehouses, or find out that the space is already booked. Furthermore, there is no point of comparison since rates vary and additional safety facilities like CCTV or fire safety are not listed unless you physically visit the location. Behind the scenes, property management remains antiquated, with booking records, client data, and revenue reports typically managed through spreadsheets. There is no existing system capable of processing complex, multi-dimensional conversational queries, such as "list warehouses in Thane for less than fifty rupees per square foot". Trust is also a significant risk, as crucial compliance documents like GST certificates, PAN, and licenses are not checked at the time of listing.

To address these critical gaps, our proposed system, SmartSpace, integrates discovery, comparison, booking, and Large Language Model (LLM) recommendations into a single platform. SmartSpace aligns warehouse seekers with their corresponding requirements by jointly processing explicit user preferences and implicit constraints extracted from the input conversation. To ensure mathematical precision, the structured recommendation engine leverages a comprehensive 5-algorithm machine learning ensemble incorporating Gradient Boosting, Neural Networks, and Hybrid Stacking alongside K-Nearest Neighbors and Random Forest. Within this ensemble, the K-Nearest Neighbors algorithm calculates the Euclidean distance between a user's target parameters and the warehouse inventory, while the Random Forest algorithm is utilized for its interpretability and ability to handle categorical features like warehouse type and amenities.

Simultaneously, the system employs the Meta Llama 3.3 70B large language model as a semantic reasoning engine. This allows the Seeker Module to use a Smart Booking Assistant which helps people search for things using natural language, re-ranks search results, and generates contextual explanations for each warehouse. To eliminate trust risks, an automated Admin Module employs a convolutional neural network to verify documents. This automated verification engine calculates a comprehensive score based on GST, PAN, Phone, Document, and Completeness. By combining these advanced technologies and ensuring high availability through a local machine learning failover strategy meaning when external connectivity is lost, the tool continues to operate using its basic machine learning models SmartSpace improves warehouse search speed by 60% and reaches a recommendation accuracy of over 85%.

CHAPTER 2

Literature Survey

The existing literature extensively examines various methods aimed at optimizing the warehousing and supply chain ecosystem, including warehouse sharing schemes, on-demand logistics, machine learning for demand forecasting, and the automation of warehouse operations. Xing et al. proposed an enhanced modeling approach for designing warehouse sharing platform systems, utilizing predictive modeling to optimize space allocation. However, their reliance on Gradient Boosting (XGBoost) demonstrated high latency during real-time inference for large datasets. In dynamic environments where users expect instantaneous search results, this latency severely limits the platform's viability. Similarly, Matusiewicz and Książkiewicz conducted a comprehensive review of shared logistics, proposing AI-driven coordination for trust and transparency; yet, their use of Random Forest algorithms struggled to handle unstructured data such as user reviews and chat logs. In a modern digital marketplace, the inability to process unstructured conversational data severely restricts user interaction. Furthermore, the application of machine learning in warehouse management has been widely reviewed by de Assis et al., who highlighted that while predictive models are excellent for inventory demand, they often act as a "black box" with exceedingly low explainability for business users. This lack of transparency prevents Small and Medium Enterprises (SMEs) from fully trusting the AI's recommendations, as they are provided numerical outputs without the necessary contextual justification required to confidently make high-stakes financial decisions.

Research into specific operational challenges, such as on-demand warehousing and retail logistics cooperation, has yielded mixed results due to algorithmic limitations. Elia et al. studied on-demand warehousing platforms to improve scalability and operational speed using Statistical Regression, but found that this method frequently fails to handle non-linear relationships in complex pricing models. Warehouse pricing is inherently multi-dimensional, influenced by localized demand, proximity to industrial hubs (like MIDC areas), and specialized amenities (such as cold chain capabilities), which basic regression cannot accurately map. Wu et al. explored a hybrid recommendation system for logistics cooperation matching using K-Nearest Neighbors (KNN); however, this approach saw performance degrade significantly when confronted with high-dimensional sparse data. In the realm of physical warehouse automation, Amoo et al. investigated AI-driven robotics and intelligent controls, developing automated systems using pattern recognition via Convolutional Neural Networks (CNNs). While highly accurate for visual inventory tracking and document analysis, these CNN architectures remain computationally expensive and require high-end GPU infrastructure, making them difficult to scale for centralized, cost-effective booking platforms.

More recent literature reflects a strong pivot toward utilizing Large Language Models (LLMs) and Natural Language Processing (NLP) to assist with decision-making, planning, and automating supply chain processes. Dhara and Barba investigated the integration of Transformer LLMs (specifically Llama 2) for enhancing supply chain customer service and planning interactions, noting that these older models are prone to "hallucinations" and lack specific domain knowledge. In logistics, hallucinatory recommendations such as suggesting a non-existent cold storage facility can lead to severe operational disruptions. Kumar demonstrated the use of NLP and neural networks for automated workflow routing and fraud detection, though traditional NLP maintains a limited contextual understanding compared to modern Generative AI. Advancing into generative systems, Min explored the potential of models like Gemini Pro to improve demand forecasting accuracy and decision support, but the strict dependency on external APIs creates severe latency and potential cost barriers that can hinder the financial scalability of the system. Finally, Qi et al. developed intelligent multi-agent systems using LLMs for automated supply chain planning under uncertain conditions, but found that the complex orchestration of these agents often leads to system instability during failovers.

Overall, while the literature robustly explores individual technologies ranging from traditional machine learning to advanced LLM agents most studies only examine one isolated aspect of the problem, such as pricing, location, or automation. The existing research critically lacks comprehensive frameworks that successfully integrate hybrid machine learning models with natural language explanations, along with sound backup plans. Current literature fails to provide a multimodal fusion architecture that can seamlessly fall back on local machine learning models when external LLM APIs fail. Most importantly, it fails to adequately address the 'cold-start' problem for newly listed warehouses that have no historical booking information, an issue that requires a sophisticated blend of feature engineering, semantic reasoning, and dynamic algorithmic weighting to overcome.

Table 1: Literature Survey Table

Sr. No.	Paper Name	Strengths	Drawbacks
1	An Enhanced Modelling Approach for Warehouse Sharing Platform System Designing Problem	Optimized warehouse space allocation using predictive modeling (XGBoost) for shared economy platforms.	High latency during real-time inference for large datasets.
2	Shared Logistics Literature Review	Analyzed logistics trends and proposed AI-driven coordination (Random Forest) for trust and transparency.	Struggles with unstructured data like user reviews or chat logs.
3	Supply Chain Management in the Era of Large Language Models	Investigated Transformer LLMs (Llama 2) for enhancing supply chain customer service and planning interactions.	Prone to "hallucinations" and lacks domain-specific knowledge.
4	AI-driven warehouse automation: A comprehensive review of robotics and intelligent controls	Developed an automated warehouse system using pattern recognition (CNNs) for inventory tracking.	Computationally expensive: requires high-end GPU infrastructure.
5	On-Demand Warehousing Platforms: A Statistical Analysis of Scalability Factors	Studied on-demand warehousing platforms to improve scalability and operational speed using Statistical Regression.	Fails to handle non-linear relationships in complex pricing models.

6	Research on Logistics Cooperation Strategy Based on Evolutionary Game Theory	Proposed a hybrid recommendation system (KNN) for retail logistics cooperation and matching.	Performance degrades significantly with high-dimensional sparse data.
7	Generative Artificial Intelligence and its Potential in Supply Chain Management: Forecasting and Decision Support	Explored using advanced LLMs (Gemini Pro) to improve demand forecasting accuracy and decision support.	Dependency on external APIs creates latency and potential cost barriers.
8	Large Language Model Agents for Automated Supply Chain Planning Under Uncertainty	Developed intelligent multi-agent systems for automated supply chain planning under uncertain conditions.	Complex orchestration often leads to system instability during failovers.
9	Transforming Logistics Technology: Workflow Automation and Fraud Detection using NLP	Demonstrated automated workflow routing and fraud detection in logistics technology using NLP.	Limited contextual understanding compared to modern Generative AI.
10	Machine learning applications in warehouse management: A systematic review	Reviewed machine learning methods for predicting inventory demand and optimizing stock levels.	Acts as a "black box" with low explainability for business users.

CHAPTER 3

Limitation of Existing System

The current warehouse ecosystem in Maharashtra suffers from several critical limitations that severely hinder operational efficiency and transparency. The landscape is incredibly diverse and fragmented, with storage facilities spread all over the region, ranging from large-scale operations in and around MIDC areas to conventional godowns, smaller galas, and dark stores utilized by rapid delivery services like Swiggy Instamart and Zepto. These facilities must cater to highly specific industry requirements, offering cold storage, pharma grade, electronics, FMCG, and general storage varieties. Despite this vast inventory, the process of finding and booking suitable warehouses is still largely manual. It relies heavily on traditional brokers and mandatory physical site visits, which leads to longer search times, poor decision making on facility selection, and higher operational costs, especially for Small and Medium Enterprises (SMEs). Finding a warehouse is fundamentally difficult because there are no centralized platforms to view all available options at one place. Because rental rates vary widely and critical additional facilities such as CCTV surveillance or fire safety equipment are often not listed online, prospective tenants have no reliable point of comparison unless they physically visit the location. This creates a massive bottleneck, as the person making the decision has to consider several factors at once, such as location, cost limitations, capacity, specialization, compliance, and quality. Prospective tenants often waste days calling brokers, visiting nearby sites, or ultimately finding out that the space they need is already booked.

Beyond the physical search experience, the administrative and technical frameworks of existing digital systems are deeply flawed. Property managers and logistics operators frequently rely on antiquated methods, managing vital booking records, client data, and revenue reports manually through basic spreadsheets. From a technical standpoint, the standard digital systems currently used to help people find warehouse space are highly ineffective because they only search for exact words and use rigid, static filters. This means they do not really understand what the user is looking for and they also have problems processing questions that are not written in a highly specific, structured way. Consequently, these platforms are entirely incapable of processing complex, natural language queries, such as a user asking to "list warehouses in Thane for less than fifty rupees per square foot". Current online solutions only address disconnected parts of this problem and lack the underlying architecture to make recommendations across multiple criteria with sound reasoning. Furthermore, existing systems are heavily penalized by the 'cold-start' problem. New listings or less frequently used warehouses tend to have insufficient historical booking data, making it exceedingly difficult for traditional collaborative filtering techniques to recommend them. As a result, newly listed warehouses suffer from poor market visibility simply due to a lack of prior user interaction data.

Finally, trust and regulatory compliance present massive risks within the current ecosystem. In existing property listing platforms, crucial government compliance documents including GST certificates, PAN cards, and operational licenses are not checked at the time of listing. This lack of upfront verification exposes businesses to potential fraud, operational disruptions, and severe legal liabilities, highlighting a critical need for automated, rigorous document verification mechanisms before a property is made visible to the public. Overcoming these limitations and showing accurate rankings for each warehouse requires highly trained models. However, traditional standalone machine learning algorithms used in the industry exhibit notable limitations, often struggling to process unstructured data or failing to handle the non-linear relationships found in complex real estate pricing models. Ultimately, these combined systemic gaps highlight the urgent need for a unified platform that integrates discovery, comparison, booking, and intelligent recommendations in one place, so that businesses do not waste time on a process that should be seamless.

Research Gap

A comprehensive review of the existing literature reveals that current online solutions and academic studies typically examine only isolated, disconnected parts of the warehousing problem. Existing literature heavily examines various specialized methods such as warehouse sharing schemes, on-demand logistics, machine learning for demand forecasting, and automating warehouse operations. Nevertheless, most of the current literature only examines one aspect of the problem, such as pricing, location, or automation alone. Consequently, current online solutions only address a disconnected part of this problem and do not have the capability to make recommendations across multiple criteria with sound reasoning. Few studies examine systems that integrate hybrid machine learning models and explanations along with sound backup plans. The absence of a unified, multi-dimensional approach forces logistics stakeholders to rely on fragmented tools that fail to capture the holistic and dynamic nature of modern supply chain planning.

A major technical challenge that remains inadequately addressed by current research is the 'cold-start' problem. Moreover, most of the existing systems face the 'cold-start' problem for new warehouses that have no historical booking information. Additionally, new listings or less frequently used warehouses tend to have insufficient data, thereby making it difficult to make sound recommendations based on traditional collaborative filtering techniques. Furthermore, standalone machine learning algorithms exhibit notable limitations when applied independently in this domain. For instance, Gradient Boosting (XGBoost) models suffer from high latency during real-time inference for large datasets. Random Forest struggles with unstructured data like user reviews or chat logs, while K-Nearest Neighbors (KNN) sees its performance degrade significantly with high dimensional sparse data. Similarly, Statistical Regression fails to handle non-linear relationships in complex pricing models. Furthermore, traditional machine learning methods often act as a "black box" with extremely low explainability for business users, providing numerical outputs without the necessary contextual justification required to confidently make high-stakes supply chain decisions.

While more recent literature indicates the increasing use of large language models (LLMs) to assist with decision-making, planning, and automating processes, there are still significant hurdles preventing widespread, reliable adoption. Many older Transformer LLMs (like Llama 2) investigated for enhancing supply chain customer service and planning interactions are prone to "hallucinations" and lack specific domain knowledge. Traditional Natural Language Processing (NLP) models paired with neural networks also demonstrate limited contextual understanding compared to modern Generative AI, making them ill-equipped to parse complex booking intents. Furthermore, an over-reliance on external APIs (such as those required for Gemini Pro) creates severe latency issues and introduces potential cost barriers that can hinder scalability. Most importantly, there is a notable absence of research addressing highly resilient, fail-safe architectures.

CHAPTER 4

Problem Statement, Objectives and Scope

4.1 Problem Statement:

The warehousing sector in Maharashtra is experiencing rapid growth, largely driven by the escalating storage demands of small businesses. Despite a diverse inventory ranging from large MIDC facilities to specialized dark stores and cold storage units, the fundamental process of searching for and booking suitable warehouse space remains archaic and largely manual. Prospective tenants are forced to rely heavily on traditional brokers and mandatory physical site visits, leading to significantly prolonged search times, suboptimal decision-making, and inflated operational costs for Small and Medium Enterprises (SMEs). Finding a warehouse is exceptionally difficult because there are no centralized platforms available to view and compare all potential options in a single location.

Beyond this manual friction, current digital solutions are fundamentally unequipped to handle the multi-dimensional complexity of warehouse selection. Standard online platforms utilize rigid, static filters and exact-word matching, which fail to capture the user's underlying intent or process complex, natural language queries such as "list warehouses in Thane for less than fifty rupees per square foot". Furthermore, existing systems are heavily penalized by the "cold-start" problem. Newly listed warehouses inherently lack historical booking data, making it exceedingly difficult for traditional collaborative filtering techniques to recommend them, thereby severely limiting their market visibility. Additionally, logistics operators and property managers are burdened by outdated administrative practices, frequently managing vital booking records and revenue reports through basic spreadsheets.

Finally, the existing ecosystem suffers from a severe lack of transparency and systemic trust. Because rental rates fluctuate and critical safety facilities such as CCTV surveillance or specialized fire safety equipment are rarely listed online, prospective tenants have no objective point of comparison unless they physically visit the location.

4.2 Objectives:

The primary objective of this project is to develop and implement SmartSpace, an intelligent, centralized warehouse recommendation system tailored specifically for the logistics environment in Maharashtra. This system aims to significantly reduce the time and operational costs associated with manual warehouse searching for Small and Medium Enterprises (SMEs) by providing a fully automated platform for property discovery, comparison, and booking. To achieve this, the project seeks to design and integrate a highly accurate hybrid machine learning ensemble incorporating algorithms such as K-Nearest Neighbors and Random Forest capable of evaluating and ranking storage facilities based on multi-dimensional criteria including location, price, capacity, and real-time availability.

Furthermore, the project intends to bridge the gap between complex mathematical ranking and user comprehension by integrating the Meta Llama 3.3 70B large language model. This specific objective focuses on enabling the system to process unstructured, natural language conversational queries, dynamically re-rank search results, and generate clear, context-aware explanations for every recommended warehouse.

Another critical objective is to establish a secure, transparent, and trustworthy logistics ecosystem. This will be accomplished by developing an automated Admin Module powered by a Convolutional Neural Network (CNN) to instantly verify essential government compliance documents, such as GST certificates and PAN cards, prior to any listing being approved for public view. Finally, the project aims to guarantee continuous, fail-safe operations by implementing a robust local machine learning fallback strategy, ensuring that the platform maintains 100% system availability and continues to provide accurate recommendations even if external connectivity to the language model is temporarily disrupted.

4.3 Scope:

The scope of this project is primarily focused on transforming the logistics and warehousing ecosystem specifically within the state of Maharashtra, India. The system's operational boundaries encompass a comprehensive dataset of over 10,000 warehouse records, which was programmatically augmented from an initial subset of certified government registry data obtained from the Warehousing Development and Regulatory Authority (WDRA). This robust dataset covers diverse facility types across multiple districts, including cold storage, pharmaceutical-grade units, and fast-moving consumer goods (FMCG) centers. The data scope incorporates 16 critically engineered attributes for each property, accounting for variables such as micro-rental space availability, occupancy rates, specific amenities, and dynamic market pricing.

Functionally, the scope of SmartSpace covers the end-to-end lifecycle of warehouse discovery, comparison, and management by dividing the application logic into three specialized modules. It includes a Seeker Module designed to process natural language queries and provide mathematically precise property recommendations, an Owner Module equipped with real-time dynamic pricing inference and inventory tracking, and an Admin Module dedicated to automated regulatory oversight. The platform is architecturally scoped to handle multimodal data inputs including images, PDFs, and standard text facilitating the seamless upload and processing of essential facility details and compliance documents. The target demographic within this scope primarily involves Small and Medium Enterprises (SMEs), property owners, and logistics operators seeking to digitize their operational workflows.

Technologically, the project scope includes the deployment of a highly resilient, dual-engine artificial intelligence architecture. This involves the implementation of a 5-algorithm machine learning ensemble utilizing models such as K-Nearest Neighbors, Random Forest, and Gradient Boosting for structured numerical property matching. Concurrently, the scope involves the integration of the Meta Llama 3.3 70B large language model via the Groq inference engine to handle semantic reasoning, intent parsing, and the generation of contextual explanations.

CHAPTER 5

Proposed System

SmartSpace follows a layered and modular approach built for scalability and reliability to handle different types of data such as Images, PDFs, and Texts. This architecture is helpful to separate structured logistics and data processing from natural language reasoning. The framework uses a React-based Presentation Layer for user interfaces, alongside a secure Application Layer created using Express.js for handling API routing and authentication. Furthermore, a Data Layer ensures secure, RLS-enabled storage utilizing Supabase PostgreSQL. The core of the system is the Intelligence Layer, which deploys a hybrid framework combining a five-algorithm machine learning ensemble for predictions and the Meta Llama 3.3 70B large language model for reasoning.

Data collecting techniques rely on a primary regulatory source to improve credibility, utilizing a dataset obtained from the Warehousing Development and Regulatory Authority (WDRA). This structured registry data initially contained 424 certified warehouse records across all districts in Maharashtra. To reflect realistic market conditions, this data was supplemented with market context features such as pricing and specific facility types. Raw government data required extensive preprocessing to transform it from static registry entries into machine learning ready feature vectors. This involved a centroid-based geospatial imputation technique to assign latitude and longitude coordinates, validation based on Regex to clean contact information, and Min-Max scaling to normalize numerical features like total area. To train the multi-dimensional recommendation engine, the initial seed data was programmatically augmented to a dataset of 10,002 records, extended to 16 critically engineered attributes, and balanced using stratified sampling to prevent algorithmic bias.

The application logic is divided into three distinct modules. The Seeker Module features an Interactive ML Recommendation Engine where users define their ideal storage profile through structured inputs to generate a query vector. Additionally, it includes a Smart Booking Assistant that implements an NLP pipeline where the LLM processes unstructured requirements and converts them into structured search queries. The Owner Module is built for managing warehouse listings and provides an Inventory Dashboard built with Recharts, alongside a Dynamic Pricing Inference tool where a pre-trained regression model outputs recommended market pricing. The Admin Module provides central control to review and authorize warehouse submissions using role-based access control. This module also deploys an automated verification engine that utilizes a weighted scoring model based on GST, PAN, Phone, Document, and Completeness parameters to evaluate listings.

To process complex queries, the system uses a dual-engine architecture that handles both matching and reasoning through multimodal fusion. The Structured Recommendation Engine utilizes a 5-algorithm ensemble incorporating K-Nearest Neighbors, Random Forest, Gradient Boosting, Neural Networks, and Hybrid Stacking. This ensemble evaluates an 8-dimensional feature space comprising location match, price fitness, area adequacy, warehouse type, verification status, availability, user rating, and amenity density to output a numerical property match. Concurrently, the Semantic Reasoning Engine uses the Meta Llama 3.3 70B model via the Groq inference engine to extract entities such as location and budget from natural language queries. The LLM applies a composite scoring function to rearrange the top 50 results from the ML model based on how well they match the user's needs. To ensure maximum reliability, if the language model is not available, the system automatically uses the local machine learning models so recommendations continue to operate without relying on any external service.

5.1 Modules:

- Seeker Module: Designed for users searching for warehouse space. It features an Interactive ML Recommendation Engine that processes structured numerical inputs (like area and budget), and a Smart Booking Assistant that uses an NLP pipeline to turn natural language requests (e.g., "I need 400 sq ft in Thane") into structured search queries.
- Owner Module: Built for warehouse operators to manage their property listings and track inventory. It includes a Dynamic Pricing Inference tool that suggests competitive rates based on real-time market data, alongside an Inventory Dashboard to monitor occupancy rates and revenue metrics.

- Admin Module: Serves as the central control hub to securely review and authorize new warehouse submissions using role-based access control. It features an automated Document Verification System powered by a convolutional neural network that calculates a trust score based on uploaded compliance documents like GST and PAN to automatically approve or reject listings.

5.2 Architecture Diagram:

The SmartSpace system architecture utilizes a secure, four-layer modular framework to process multimodal logistics data. The Presentation Layer features a React-based frontend providing tailored web interfaces such as search tools, AI chatbots, and pricing dashboards for seekers, owners, and admins. Below this, the Application Layer acts as a secure API gateway, using JWT and Role-Based Access Control (RBAC) to safely route system requests. The core Intelligence & Services Layer powers the platform with a dual-AI engine: a 5-algorithm machine learning ensemble handles numerical recommendations and document verification, while the Meta Llama 3.3 70B model manages natural language reasoning. Finally, the Data Layer ensures secure information storage via a Row-Level Security (RLS) enabled Supabase PostgreSQL database, which is integrated with a data preprocessing pipeline and official WDRA regulatory data.

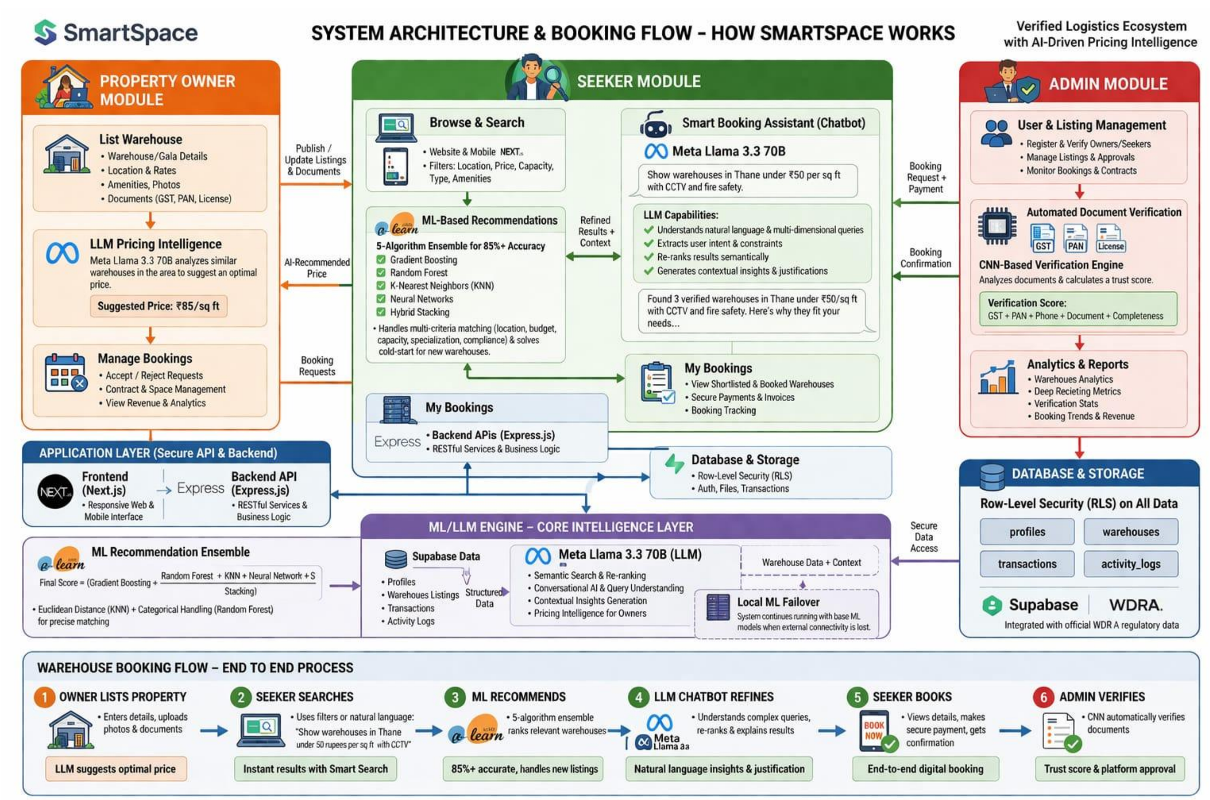


Fig 1: Architecture Diagram

5.3 UML Diagrams:

A UML diagram is a diagram based on the UML (Unified Modeling Language) to visually represent a system along with its main actors, roles, actions, artifacts, or classes, to better understand, alter, maintain, or document information about the system.

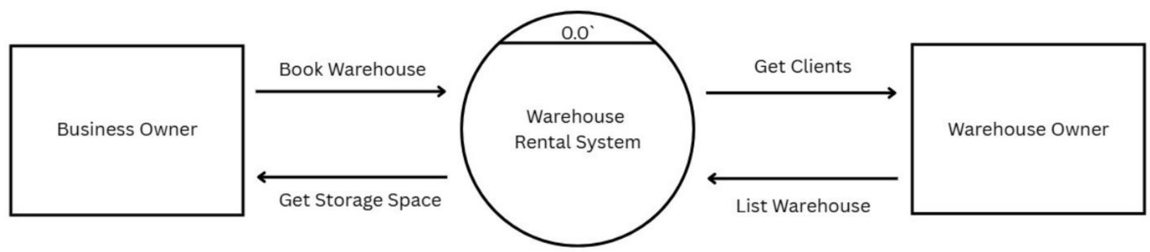


Fig 2: Data Flow Diagram Level 0

The figure illustrates the Level 0 Data Flow Diagram (Context Diagram) for the SmartSpace platform, providing a high-level overview of the warehouse rental system's primary data flows. The central process, designated as "0.0 Warehouse Rental System," serves as the core platform connecting the two main external entities: the Business Owner and the Warehouse Owner. As depicted in the diagram, the Business Owner interacts with the system by initiating a "Book Warehouse" request, and in return, they achieve their goal to "Get Storage Space." Conversely, the Warehouse Owner provides data to "List Warehouse" on the platform, which the system subsequently utilizes to help them "Get Clients". This top-level view concisely captures the fundamental interaction loop where the system integrates property listing, discovery, and booking to serve both space seekers and property managers efficiently.

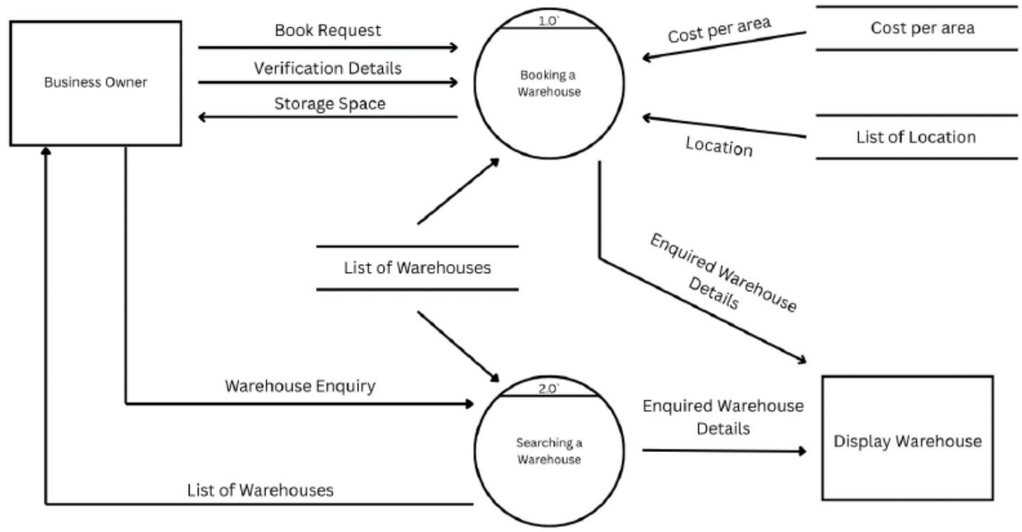


Fig 3: Data Flow Diagram Level 1

The figure illustrates the Level 1 Data Flow Diagram for the SmartSpace platform, providing a detailed breakdown of the primary operational processes from the perspective of the Business Owner. The system's functionality is divided into two core sub-processes:

In process **"2.0 Searching a Warehouse,"** the workflow initiates when the Business Owner submits a "Warehouse Enquiry." The system processes this input by querying the "List of Warehouses" data store. It then returns a filtered "List of Warehouses" back to the user and simultaneously sends the "Enquired Warehouse Details" to the "Display Warehouse" interface for visual confirmation.

Once a facility is selected, the user engages with process **"1.0 Booking a Warehouse"** by submitting a "Book Request" alongside the necessary "Verification Details." To validate and process this transaction, the system pulls specific data from multiple data stores, including "Cost per area," "List of Location," and the main "List of Warehouses." Upon successful processing, the system outputs the finalized "Enquired Warehouse Details" to the display and officially allocates the requested "Storage Space" back to the Business Owner.

This diagram effectively demonstrates the seamless data transition between the search and booking phases, highlighting how various data stores are accessed to validate user requests.

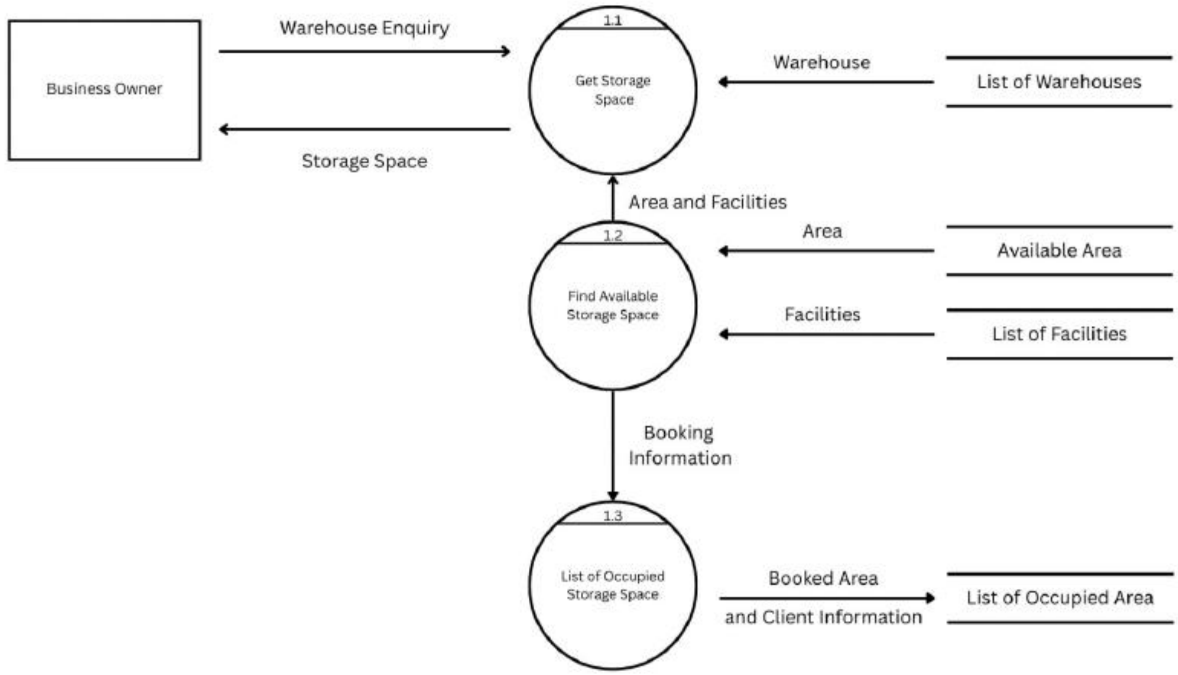


Fig 4: Data Flow Diagram Level 2

The figure illustrates the Level 2 Data Flow Diagram, detailing the specific sub-processes involved when a Business Owner queries and secures warehouse space. This diagram breaks down the internal mechanics of the booking phase:

The sequence begins with process **1.1 Get Storage Space**, which receives a "Warehouse Enquiry" from the Business Owner and retrieves general property data from the "List of Warehouses" data store.

To determine exact capacity and amenities, process **1.2 Find Available Storage Space** extracts specific "Area" metrics from the "Available Area" data store and "Facilities" data from the "List of Facilities" data store. This combined "Area and Facilities" information is then sent back to process 1.1 to validate the user's request against real-time availability.

Upon successful matching and confirmation, process 1.2 forwards the final "Booking Information" to process **1.3 List of Occupied Storage Space**. This process acts as the inventory updater, permanently recording the "Booked Area and Client Information" into the "List of Occupied Area" data store to prevent future double-booking. Finally, process 1.1 completes the cycle by officially confirming and assigning the "Storage Space" back to the Business Owner.

5.4 Sequence Diagram:

The Sequence Diagram for the SmartSpace platform, mapping the chronological flow of interactions during a warehouse booking operation. The sequence initiates when a Business User submits a "Request storage" message to the central SmartSpace Platform, detailing their specific logistical requirements. Upon receiving this, the platform communicates with the Warehouse Provider to "Check available capacity." This triggers the provider module to query the Real Estate Database to "Fetch real estate data," retrieving up-to-date property details and current occupancy rates to ensure only vacant spaces are considered.

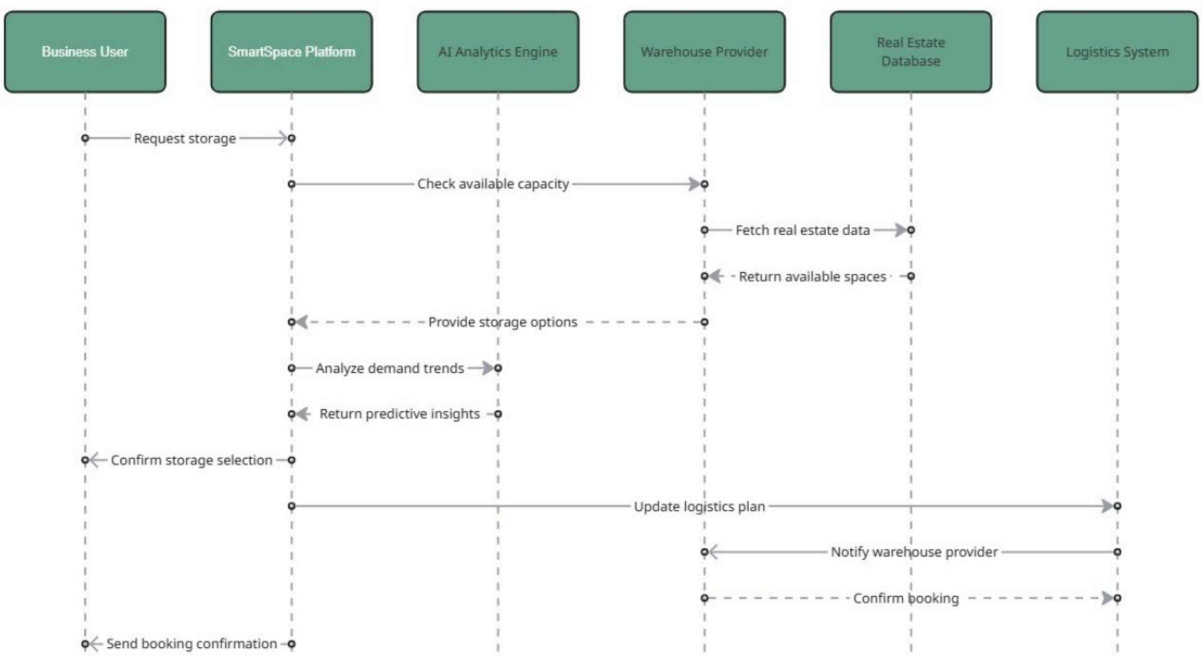


Fig 5: Sequence Diagram

Once the data regarding available storage spaces is relayed back, the SmartSpace Platform initiates its intelligent processing phase. It engages the AI Analytics Engine, sending a request to "Analyze demand trends." The AI evaluates the available inventory against real-time market data, pricing models, and user constraints, processing this information to "Return predictive insights." This crucial step ensures that the platform filters out suboptimal choices and prepares a highly refined, intelligently ranked list of storage options tailored exactly to the user's needs.

5.5 Activity Diagram:

The Activity Diagram for the system, mapping the dynamic user workflow, parallel processes, and decision-making checkpoints. The flow originates at the initial state and branches into two primary operational tracks. The first track governs user authentication and communication. Here, the user enters the "Login" state, followed by a "Verify" decision node. If the credentials are "invalid," the flow loops back to prompt another login attempt. Upon a "valid" verification, the user proceeds to the "Check Mails" action to view system communications before moving toward the final synchronization phase.

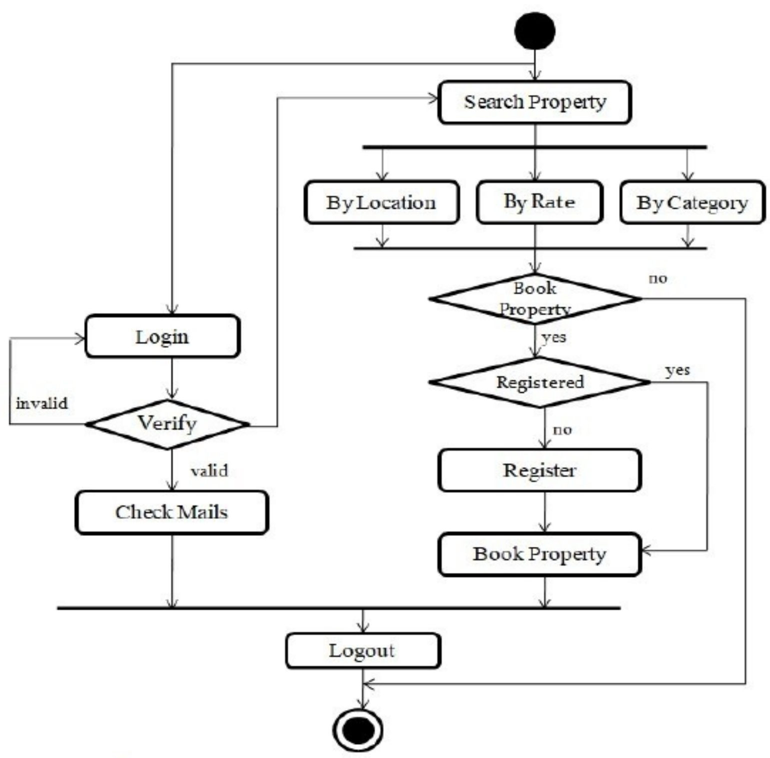


Fig 6: Activity Diagram

Concurrently, the core business track begins with the "Search Property" action, utilizing parallel filters for location, rate, and category. Following a "Book Property" decision, users opting to proceed are checked for registration status. Unregistered users must "Register" before finalizing their booking, while registered users bypass this step. Finally, both the authentication and booking tracks converge at a synchronization bar.

CHAPTER 6

Experimental Setup

The experimental setup for the SmartSpace platform was designed to effectively evaluate the dual-engine AI architecture, ensuring high-speed natural language inference alongside mathematically precise machine learning recommendations. The configuration encompasses specific hardware parameters, software frameworks, and data environments necessary to replicate and validate the proposed system's performance.

6.1.1 Dataset and Preprocessing Environment:

The foundation of the experimental setup is built upon a certified government registry dataset obtained from the Warehousing Development and Regulatory Authority (WDRA), initially containing 424 records across Maharashtra. To simulate realistic, large-scale market conditions, this data was programmatically augmented to a comprehensive dataset of 10,002 warehouse records. The preprocessing pipeline was executed using Python, utilizing the Pandas and NumPy libraries. Key preprocessing steps included centroid-based geospatial imputation for mapping missing latitude and longitude coordinates, Regex validation for sanitizing contact information, and

Min-Max scaling to normalize continuous variables like total area and price. The final dataset utilized for training and inference consists of 16 critically engineered attributes, balanced using stratified sampling to prevent algorithmic bias.

6.1.2 Software and Development Frameworks:

The platform's architecture spans multiple technological stacks to handle both web operations and intelligent processing:

- **Web Frameworks:** The Presentation Layer was developed using React.js to render user-specific interfaces and the Recharts library for the dynamic owner dashboard. The Application Layer operates on a Node.js environment using the Express.js framework for robust API routing and JWT-based authentication.
- **Database:** Supabase PostgreSQL was implemented as the primary Data Layer, configured with Row-Level Security (RLS) to manage structured warehouse records, user credentials, and booking histories securely.
- **Machine Learning Libraries:** The structured recommendation engine and document verification modules were developed in Python. Scikit-learn was utilized to construct the 5-algorithm ensemble (incorporating K-Nearest Neighbors, Random Forest, and Gradient Boosting), while TensorFlow/Keras was employed to build and train the Convolutional Neural Network (CNN) for the Admin Module's automated document verification.

6.1.3 Hardware and Inference Infrastructure:

To handle the computational demands of the system without introducing latency, the hardware setup was strategically divided between local processing and cloud-based inference:

- **Semantic Reasoning Engine**: To run the highly resource-intensive Meta Llama 3.3 70B large language model, the system leverages the Groq inference engine. By utilizing Groq's specialized Language Processing Units (LPUs), the system achieves near-instantaneous natural language parsing and intent extraction without the need for expensive local GPU clusters.
- **Local ML Fallback & Execution:** The 5-algorithm machine learning ensemble and the CNN are designed to be lightweight enough to run on standard virtual private

servers (VPS) or local compute environments (e.g., standard multi-core x86 CPUs with basic GPU acceleration). This setup ensures the crucial fail-safe mechanism remains operational; if connectivity to the Groq API is lost, the local machine learning models seamlessly take over the recommendation pipeline to guarantee continuous system availability.

6.1.4 Model Configuration:

Parameters During the experimental phase, specific algorithmic parameters were configured to optimize accuracy. The K-Nearest Neighbors (KNN) model was set to utilize Euclidean distance to measure the spatial and parametric proximity between user requirements and warehouse attributes. The Random Forest classifier was configured to handle categorical data (like warehouse type and amenity density) to maintain high interpretability. For the document verification process, the CNN was configured to accept standardized, resized image inputs (e.g., uploaded GST and PAN PDFs converted to image arrays) to output a weighted trust score based on document clarity and completeness.

6.2 Software and Hardware Setup

Hardware Requirements

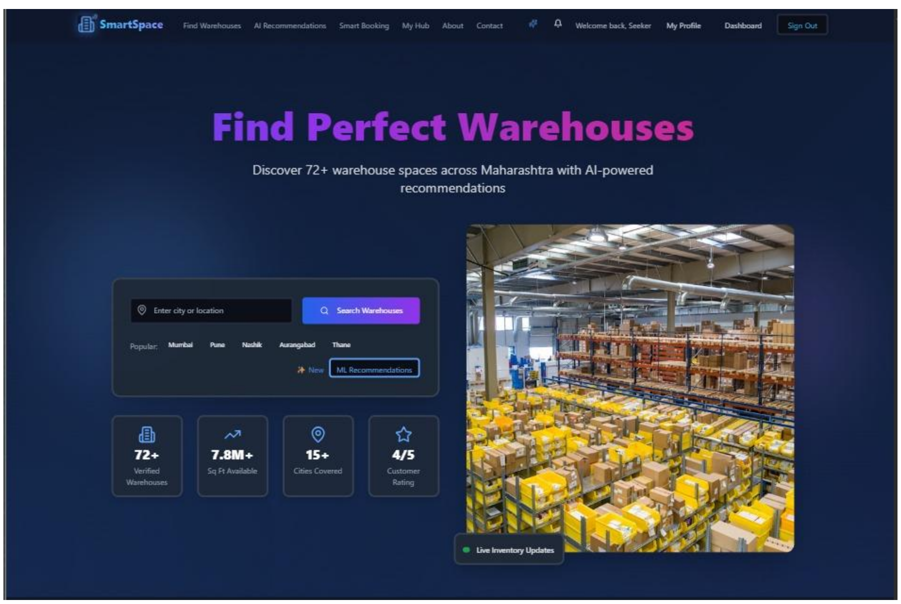
- Processor (CPU): Intel Core i5 / i7 (8th Gen or above) or AMD Ryzen 5 / 7 equivalent
- Memory (RAM): 16 GB DDR4 (Minimum) / 32 GB (Recommended)
- Storage: 256 GB SSD (Minimum)
- Graphics (GPU): Dedicated GPU with 4GB+ VRAM (e.g., NVIDIA GTX 1650, RTX 3050, or equivalent)
- Network: High-speed broadband internet connection

Software Requirements

- Operating System: Windows 10/11, macOS (Monterey or later), or Linux (Ubuntu 20.04 LTS+)
- Development Environment: Visual Studio Code (VS Code) or PyCharm
- Version Control: Git, GitHub
- API Testing: Postman
- Frontend Stack: React.js, Recharts, HTML5, CSS3, JavaScript/ES6+
- Backend Stack: Node.js (v18.x or higher), Express.js
- Database: Supabase (PostgreSQL)
- Programming Languages: Python 3.9+ (for AI/ML components)
- Data Processing: Pandas, NumPy
- Machine Learning & Deep Learning: Scikit-learn, TensorFlow or Keras
- **LLM Integration:** Groq API (for Meta Llama 3.3 70B)

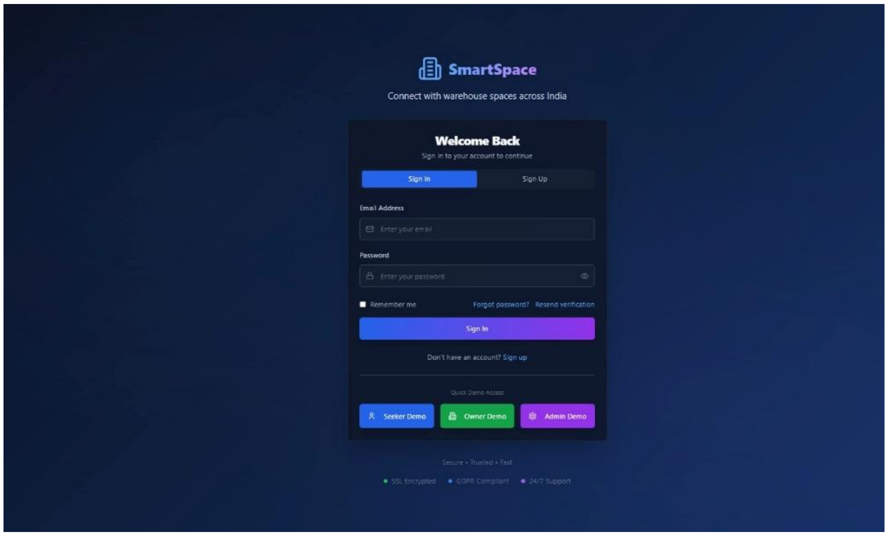
CHAPTER 7

Implementation and Result



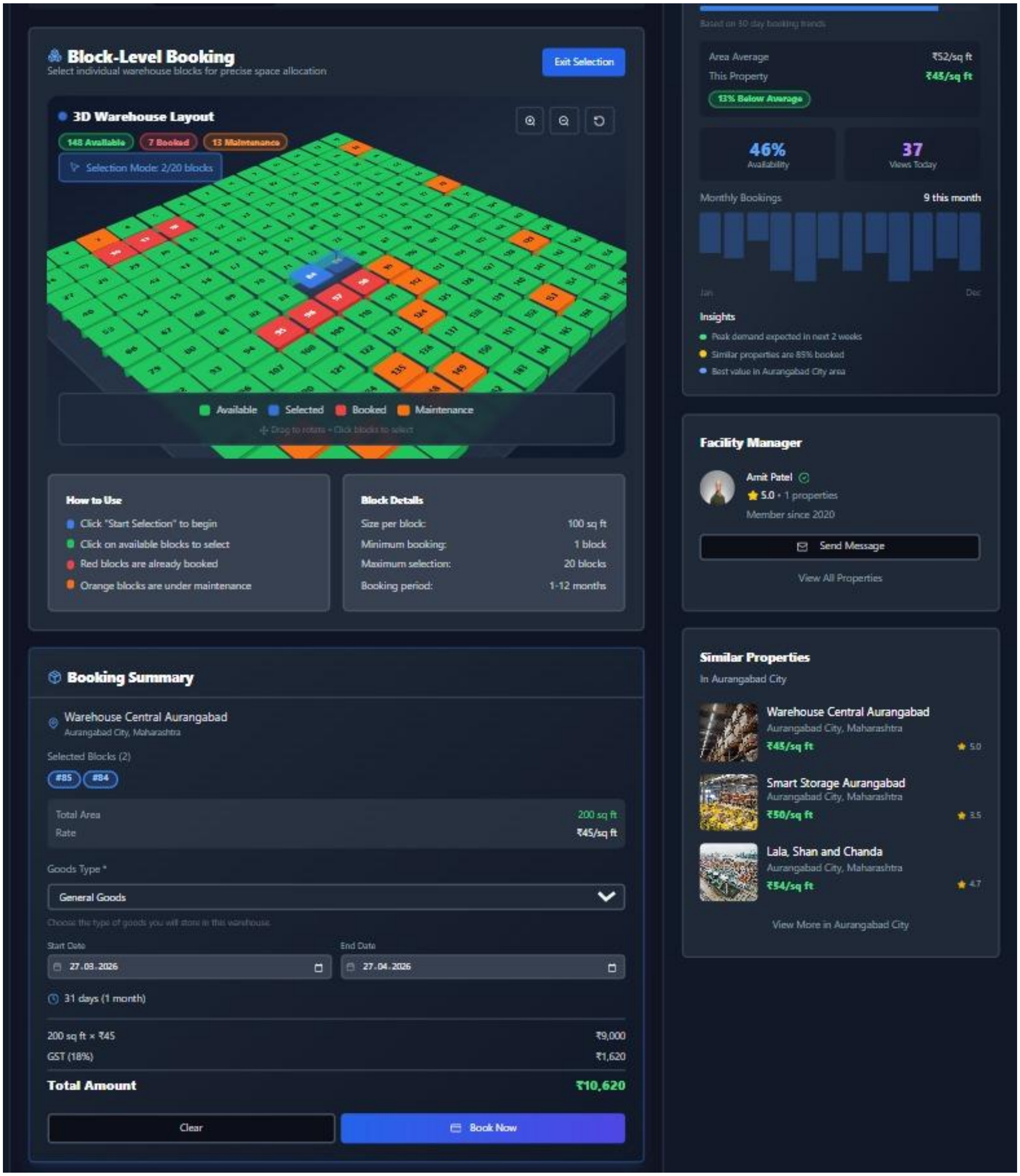
Screenshot 1: Landing Page

The first page the user will see before using the actual system



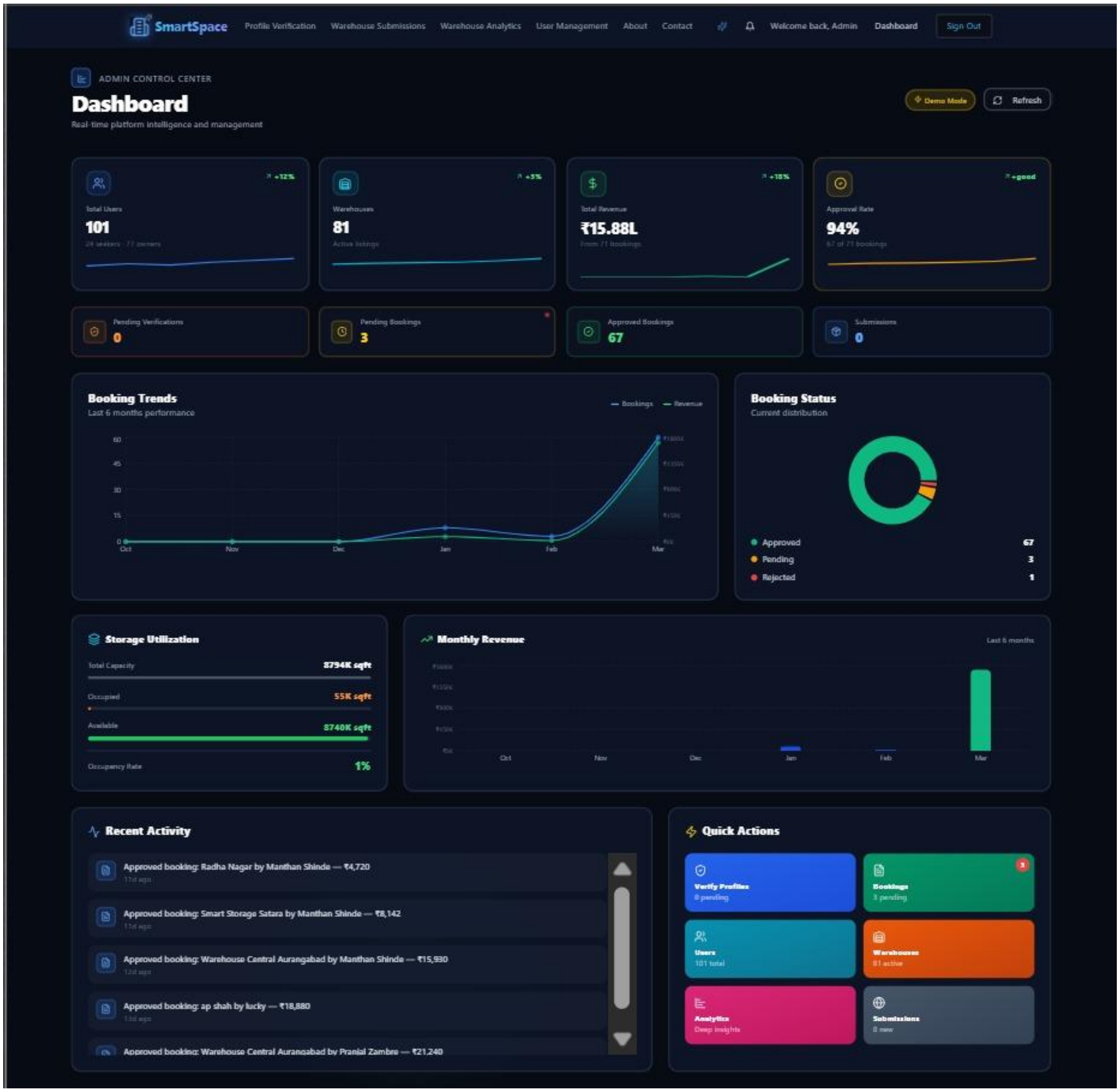
Screenshot 2: Authentication Page

The Login / Signup Page where different types of Users authenticate to access their respective Systems



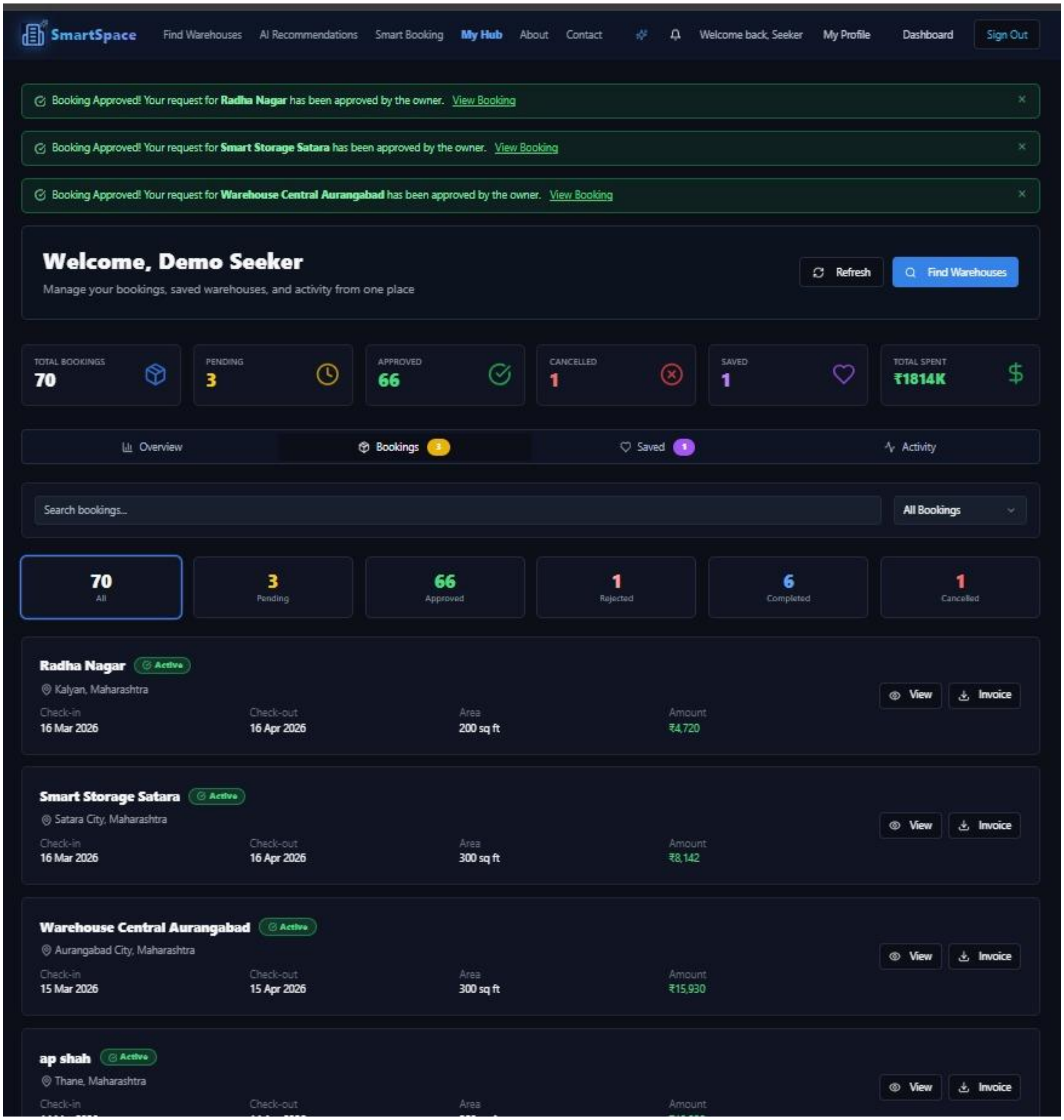
Screenshot 3: Warehouse Booking

The Page where Seeker will book the specific blocks of the Warehouse for a specific amount of time.



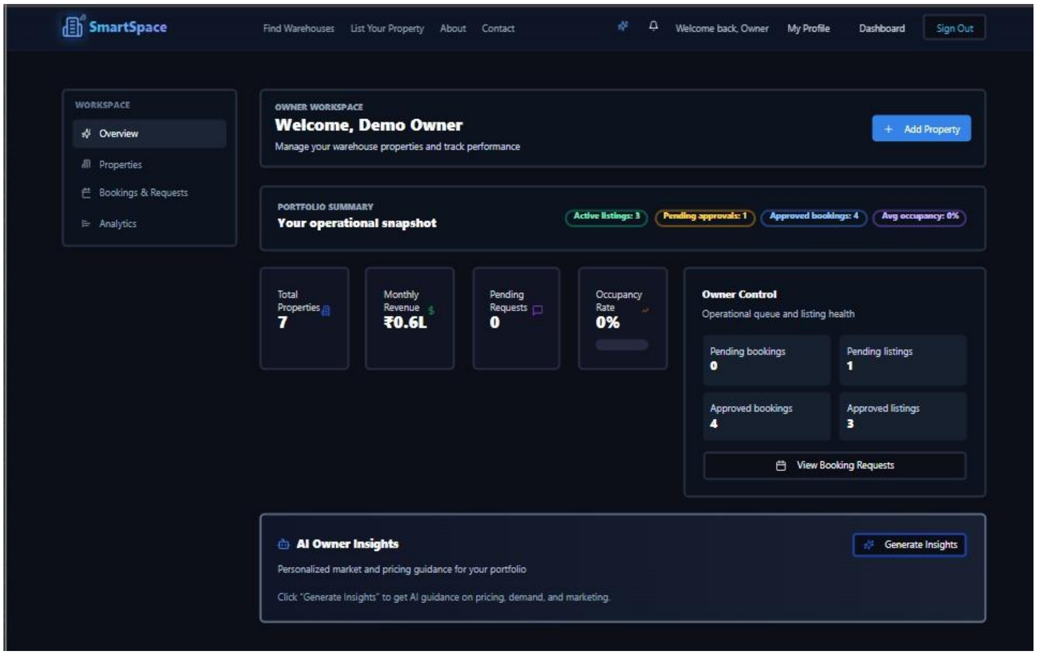
Screenshot 4: Admin Page

The Page where Admins of the System can manage the platform by approving or rejecting bookings, approving a new Warehouse Listing etc.



Screenshot 5: Seeker Page

The Page where Seeker will manage the existing bookings of the Warehouse and view invoices and view Warehouse Listings.



Screenshot 6: Owner Page

The page where Warehouse Owners can manage their Warehouse and see their Warehouse Statistics.

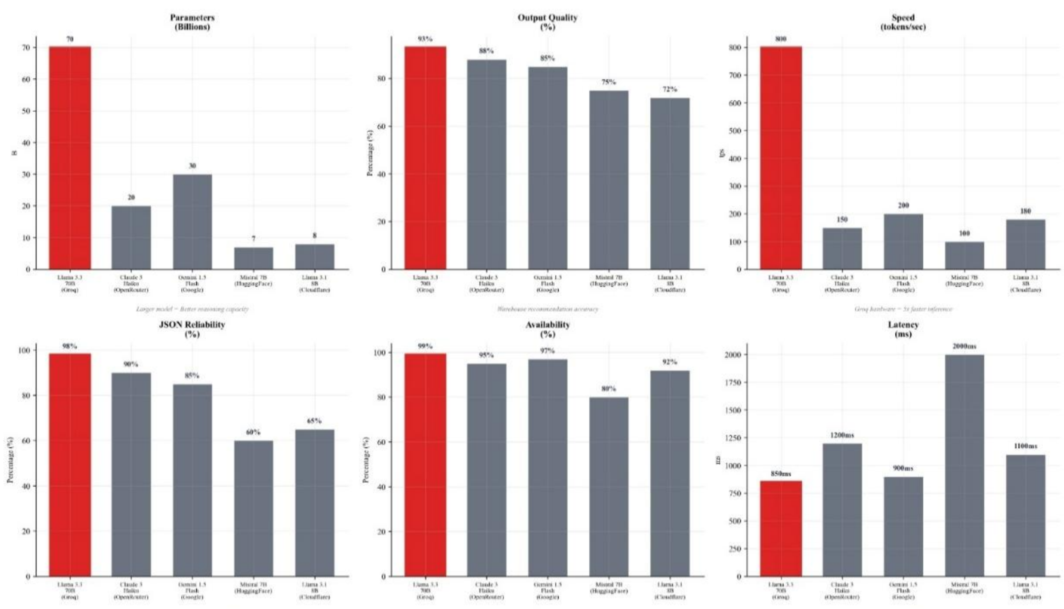


Figure 7: ML v/s LLM Comparison Chart

CHAPTER 8

Project Plan

The project plan for the SmartSpace platform follows a structured agile methodology, ensuring iterative development, continuous integration, and systematic testing of both the web infrastructure and the dual-engine AI architecture. The project lifecycle is divided into six distinct phases, spanning an estimated development period of 19 weeks.

Phase 1: Initiation and Requirement Analysis (Estimated Duration: 2 Weeks) This initial phase focuses on establishing the foundational market context and securing the raw data required to train the recommendation engines. Key activities include problem formulation, conducting an extensive literature survey, defining the project scope, and acquiring the initial certified dataset from the WDRA government portal. The primary milestone is finalizing the 16 engineered attributes needed for the warehouse data.

Phase 2: System Design and Architecture (Estimated Duration: 3 Weeks) This phase involves transforming the conceptual requirements into a concrete technical roadmap. Key deliverables include blueprinting the system layout by drafting UML diagrams (such as DFD, Sequence, Activity, and ER diagrams), designing the database schema, and creating UI/UX wireframes for the Seeker, Owner, and Admin modules. The 4-layer architecture (Presentation, Application, Intelligence, Data) is strictly defined here to prevent scope creep during development.

Phase 3: Core Application Development (Estimated Duration: 4 Weeks) This represents the traditional software engineering phase where the core functionality is built. Activities include setting up the Supabase PostgreSQL database with Row-Level Security (RLS), and developing the React.js Presentation Layer alongside the Node.js and Express.js Application Layer. The focus is on implementing core CRUD operations, user dashboards, secure backend routing, JWT authentication, and role-based access control to create a secure, functional baseline platform.

Phase 4: AI and Machine Learning Integration (Estimated Duration: 5 Weeks) As the most computationally intensive phase, this involves bridging the static web platform with the Intelligence Layer. Key tasks include data preprocessing and augmentation to scale the dataset to over 10,000 records. The team will train the 5-algorithm ML ensemble and the document verification CNN, while also integrating the Meta Llama 3.3 70B model via the Groq inference API. The goal is to establish the multimodal fusion setup, ensuring the NLP chatbot seamlessly feeds structured parameters into the ML recommendation engine.

Phase 5: System Testing and Quality Assurance (Estimated Duration: 3 Weeks) This phase goes beyond standard bug tracking to ensure system robustness. Activities include conducting unit, integration, and load testing. The team will strictly evaluate the automated failover mechanism to ensure local ML models reliably take over if the Groq API fails. Additional focus will be placed on adversarial testing of the LLM to prevent "hallucinations," tuning the prompt engineering for the NLP pipeline, and benchmarking the latency of the CNN document verification module.

Phase 6: Deployment and Documentation (Estimated Duration: 2 Weeks) The final phase centers on system handover and launch. Activities include finalizing system deployment on cloud infrastructure or a virtual private server (VPS). The team will compile the final project report, user manuals, presentation materials, and research paper drafts (if applicable), readying the application for real-world user onboarding and the final academic review.

Gantt Chart

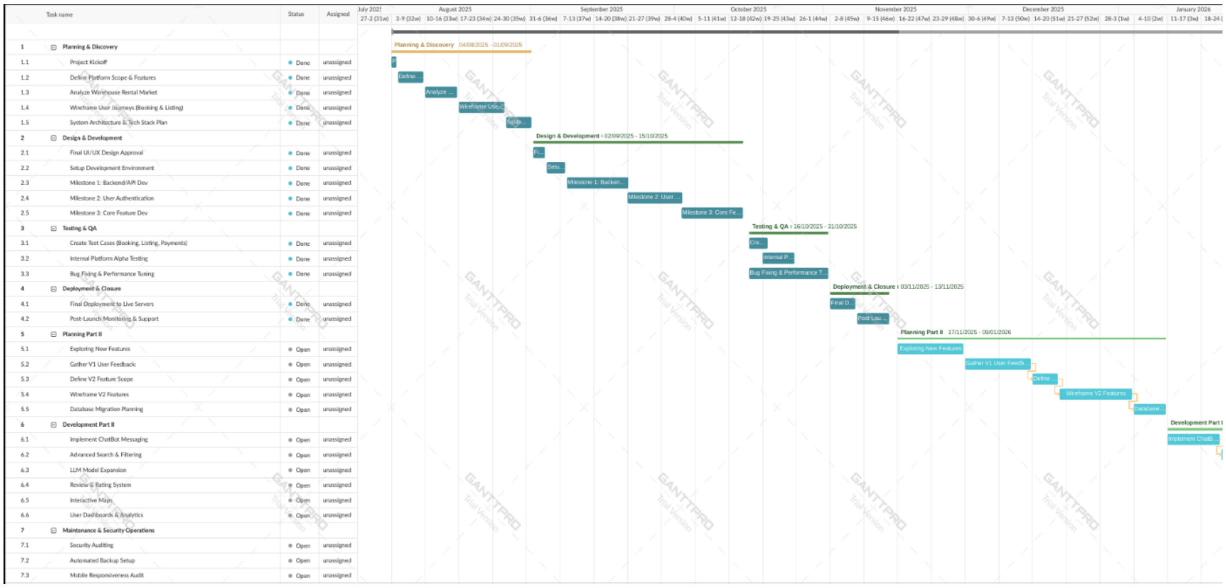


Figure 8: Gantt Chart Part 1

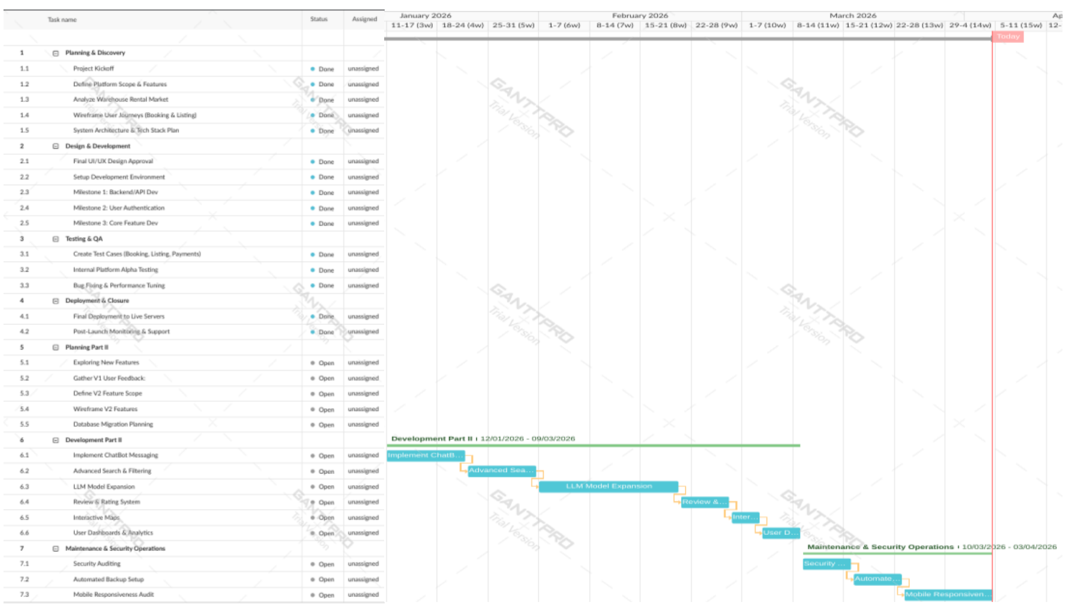


Figure 9: Gantt Chart Part 2

CHAPTER 9

Conclusion

The SmartSpace platform successfully addresses the critical inefficiencies and fragmentation present within the modern warehousing and logistics ecosystem. By developing a centralized, artificially intelligent marketplace, this project bridges the gap between Small and Medium Enterprises (SMEs) seeking flexible storage and property owners looking to optimize their facility occupancy. Moving beyond traditional, static listing platforms, SmartSpace introduces a highly dynamic and responsive environment where complex logistical requirements are instantly matched with verified properties across Maharashtra, utilizing a robust dataset of over 10,000 augmented records.

Technologically, the project represents a significant step forward in applied supply chain logistics through its implementation of a dual-engine AI architecture. By seamlessly integrating a 5-algorithm machine learning ensemble with the Meta Llama 3.3 70B large language model, the system successfully resolves the "black box" problem inherent in traditional predictive models. Users are provided not only with mathematically precise property matches but also with contextual, natural language explanations for those recommendations. Furthermore, the strategic use of the Groq inference engine mitigates the high latency typically associated with generative AI, while the engineered local fallback mechanism guarantees uninterrupted service, ensuring the platform remains resilient even during external API failures.

Ultimately, SmartSpace streamlines the entire warehouse rental lifecycle, from natural language discovery to booking and regulatory verification. Features such as the automated Convolutional Neural Network (CNN) for document verification and the Dynamic Pricing Inference tool empower administrators and warehouse owners to operate with unprecedented efficiency and market awareness. By successfully merging structured data analytics with advanced semantic reasoning, this project delivers a scalable, secure, and highly intelligent solution poised to modernize warehouse sharing and on-demand logistics.

References

[1] Z. Xing, Y. Wang, and J. Liu, “An Enhanced Modelling Approach for Warehouse Sharing Platform System Designing Problem,” arXiv preprint arXiv:2401.15389, 2024.

[2] M. Matusiewicz and D. Ksiażkiewicz, “Shared Logistics Literature Review,” Applied Sciences, vol. 13, no. 4, p. 2352, 2023.

[3] S. Dhara and S. D. Barba, “Supply Chain Management in the Era of Large Language Models,” Master’s thesis, Politecnico di Milano, Milan, Italy, 2023.

[4] O. Amoo, J. Smith, and A. Patel, “AI-driven warehouse automation: A comprehensive review of robotics and intelligent controls,” GSC Advanced Research and Reviews, vol. 18, no. 2, pp. 112–125, 2024.

[5] V. Elia, M. G. Gnoni, and F. Tornese, “On-Demand Warehousing Platforms: A Statistical Analysis of Scalability Factors,” Logistics, vol. 8, no. 4, p. 45, 2024.

[6] J. Wu, L. Zhang, and X. Chen, “Research on Logistics Cooperation Strategy Based on Evolutionary Game Theory,” Sustainability, vol. 16, no. 8, p. 3421, 2024.

[7] H. Min, “Generative Artificial Intelligence and its Potential in Supply Chain Management: Forecasting and Decision Support,” Logistics, vol. 9, no. 1, 2025.

[8] Y. Qi, R. Li, and T. Huang, “Large Language Model Agents for Automated Supply Chain Planning Under Uncertainty,” arXiv preprint arXiv:2501.03211, 2025.

[9] M. Kumar, “Transforming Logistics Technology: Workflow Automation and Fraud Detection using NLP,” International Journal of Computer Trends and Technology, vol. 72, no. 1, pp. 55–62, 2025.

[10] R. F. de Assis, R. M. Souza, and P. C. Silva, “Machine learning applications in warehouse management: A systematic review,” Procedia Computer Science, vol. 232, pp. 145–154, 2024.

Annexure-I:

Project Cost Estimation

Category of the Software Project: Semi-Detached **Considering Effort Adjustment Factor (EAF):** 1 **LOC (Lines of Code):** 1.512 (Project encompasses Python, React, and Node.js) **KLOC:** 1.512

COCOMO Model Calculations: For a Semi-Detached Project, the constants are: $a = 3.0, b = 1.12, c = 2.5, d = 0.35$

1. Effort (E): E = $a * (KLOC)^b$ * EAF => E = $3.0 * (1.512)^{1.12} * 1$ ***E* = 4.77 P-M (Person-Months)**

2. Time (T): $T = c * (E)^d$ $T = 2.5 * (4.77)^{0.35}$ ***T* = 4.32 Months**

3. People (P): P = $\frac{Effort\ (E)}{Time\ (T)}$ P = $\frac{4.77}{4.32}$ **P = 1.12 Persons**

Cost Estimation Breakdown

1. Average Monthly Developer Salary: = Rs. 15,000/- per month

2. Developer Cost: = T (time in months) × Average Monthly Salary × (No. of members) = 4.32 × Rs. 15,000/- × 3 = **Rs. 194,325/-**

3. System Cost: = Cost of 1 Machine × (No. of members) = Rs. 55,000/- × 3 = Rs. 165,000/-

Final System Cost: = {System Cost – (75% of System Cost)} + Additional H/W Cost = {165,000 - 123,750} + 0 = **Rs. 41,250/-**

4. Paid Software Costing: (e.g., Supabase Free Tier, Open-Source ML Libraries, Free Groq API Tier) = **Rs. 0/-**

5. Miscellaneous Costs: (e.g., Domain registration, documentation, buffer) = **Rs. 10,000/-**

Total Project Cost: = Developer Cost (2) + Final System Cost (3) + Paid Software Costing (4) + Miscellaneous Costs (5) = 194,325 + 41,250 + 0 + 10,000 **Total Cost = Rs. 245,575/-**

Publication

The research paper titled "Intelligent Warehouse Recommendation for Maharashtra Using Hybrid Machine Learning and LLM Assistance" has been submitted to the CIMA 2026 conference (https://scrs.in/conference/CIMA2026), and is currently under review.

Intelligent Warehouse Recommendation for Maharashtra Using Hybrid Machine Learning and LLM Assistance

Varsha Wangikar[1*], Dipesh Sharma[2], Manthan Shinde[2], Pranjal Zambre[2], Sahil Shinde[2]

[1*]Assistant Professor, Department of Computer Engineering, A.P. Shah Institute of Technology, Thane, India.
[2]Student, Department of Computer Engineering, A.P. Shah Institute of Technology, Thane, India.

*Corresponding author(s). E-mail(s): vmwangikar@apsit.edu.in;
Contributing authors: dipeshsharma34@apsit.edu.in; manthanshinde120@apsit.edu.in; pranjalzambre115@apsit.edu.in; sahilshinde13@apsit.edu.in;

Abstract

SmartSpace is a system that helps users to identify warehouses in Maharashtra, India. The system helps the user by using data from over 10,000 warehouses, reducing the time spent on manually searching and selecting the warehouses by providing an automated solution. Here system uses hybrid machine learning algorithm that combines use of K-Nearest Neighbors and Random Forest algorithm to provide accurate warehouse recommendations. The system recommends warehouses by considering parameters such as location, price, warehouse area, facility type, real-time availability and user requirements. The system uses the

Meta Llama 3.3 70B large language model to generate explanation of each warehouse. When external connectivity is lost, the tool continues to operate using its basic machine learning models. Created using Supabase PostgreSQL, React with TypeScript and Express.js. Our system improves warehouse search speed by 60% and reaches a recommendation accuracy of over 85%. This shows that our system is an scalable and efficient solution for a modern logistics environment.

Keywords: warehouse recommendation, hybrid machine learning, random forest, K-nearest neighbors, large language models

1 Introduction

1.1 Current Limitations in Warehouse Ecosystem

There are warehouses in Maharashtra all over the place. There are large stores in and around MIDC areas, conventional dungeons, smaller galas, and dark stores for fast delivery. They come in cold storage, pharmaceutical grade, electronics, FMCG, and general storage varieties. SmartSpace partners with all of them. However, the process of finding and booking warehouses is currently time consuming. Finding warehouse is difficult with no other particular platforms to view all available warehouses at one place. It takes days to call up brokers, visit nearby warehouses, or find out that the space is already booked. There is no point of comparison since rates vary and additional facilities such as CCTV or fire safety are not listed unless you physically visit the location.

1.2 Technological Advancement With LLM

A Large Language Model is a computer program that learns from a large amount of text, so that it can recognize and understand human language. The normal systems used to help people find things in warehouses are not very effective because they only search for words and use filters that do not change, which means they do not really understand what the user is looking for and they also have problems with questions that are not written in a certain way. Our system uses Meta Llama 3.3 70B in all its components to eliminate these problems, and the Seeker Module uses Smart Booking Assistant which helps people search for things using natural language, for example, you can ask it to "Show warehouses in Mumbai under Rs. 80/sq ft" and it will give you what you are looking for [4, 7]. The Owner Module uses LLM to write descriptions of properties and to give suggestions on prices based on what is happening in the market [3]. The Admin Module uses convolutional neural network to verify documents.

2 LITERATURE SURVEY

This section discusses the current literature on warehouse management systems, shared logistics and AI in supply chains. The existing literature examines various methods such as warehouse sharing schemes, on-demand logistics, machine learning for demand forecasting and automating warehouse operations [2, 5, 10]. More recent literature indicates the increasing use of large language models (LLMs) to assist with decision-making, planning, and automating processes [3, 4]. Nevertheless, most of the current literature only examines one aspect of the problem, such as pricing, location, or automation alone. Few studies examine systems that integrate hybrid machine learning models and explanations along with sound backup plans. Table I summarizes the major algorithms and constraints of the above studies, which helped inform our proposed system. Moreover, most of the existing systems face the 'cold-start' problem for new warehouses that have no historical booking information, which is a problem our research addresses through feature filtering [1].

3 Design

SmartSpace follows a layered and modular approach built for scalability and reliability to handle different types of data such as Images, PDFs, Texts etc [6]. This architecture is helpful to separate structured logistics and data processing from natural language reasoning. The overall framework consists of the following major stages:

- **Data Collection:** Collecting government-certified warehouse registry data from official government sources.
- **Data Preprocessing:** Cleaning and performing synthetic augmentation to create a market representative dataset.
- **Model Selection and Training:** Implementing a 5-algorithm machine learning model for numerical recommendations and also a Large Language Model (LLM) which is used for semantic reasoning [9].
- **Multimodal Fusion:** Integrating explicit preference vectors that a user wants with implicit conversational intent to generate a recommendations which is based on a rank.

The architecture uses a React based Presentation Layer for user interfaces and a secure Application Layer for handling API routing and authentication of different types of users. The core Intelligence Layer uses a hybrid framework, Meta Llama 3.3 70B for reasoning with an ML ensemble (KNN, RF, GB, NN, and Stacking) for predictions. A Data Layer is used to ensure secure, RLS-enabled storage via Supabase PostgreSQL and integrates external WDRA regulatory data [6, 8].

3.1 Data Collection

Data collecting techniques often relies on a primary regulatory sources to improve credibility. The orignal dataset was obtained from the *Warehousing Development and Regulatory Authority* (WDRA), a statutory body which comes under the Government of India.

- **Structured Registry Data:** This raw dataset has 424 certified warehouse records across all the districts in Maharashtra. The Key attributes of a warehouse include warehouse capacity (in Metric Tons), location, registration validity, and contact details of the owner.

Table 1: Literature Survey

Paper No.	Description of Paper	Algorithm Used	Challenges / Limitations
[1]	Optimized warehouse space allocation using predictive modeling for shared economy platforms.	Gradient Boosting (XGBoost)	High latency during real-time inference for large datasets.
[2]	Analyzed logistics trends and proposed AI-driven coordination for trust and transparency.	Random Forest	Struggles with unstructured data like user reviews or chat logs.
[3]	Explored using LLMs to support simulation-based optimization for complex supply chain tasks.	LLM assisted simulation optimization	Requires high-quality synthetic data for accurate simulation.
[4]	Investigated decentralized warehouse management using LLM agents for autonomous resource allocation.	Multi agent LLM with decision theory	Performance depends heavily on prompt clarity and context length.
[5]	Studied on-demand warehousing platforms to improve scalability and operational speed.	Statistical Regression	Fails to handle non-linear relationships in complex pricing models.
[6]	Provided a technical report on building full-stack logistics applications using modern web frameworks.	React /Express /Supabase	Scalability is limited by database connection pooling in high-traffic scenarios.
[7]	Analyzed LLMs for knowledge integration and complex reasoning within supply chain management.	Retrieval Augmented Generation (RAG)	High computational cost for real-time complex reasoning tasks.
[8]	Proposed "Warehousing 5.0" as an intelligent and adaptive ecosystem for future logistics.	Intelligent Industry 5.0 adaptive systems	Integration with legacy physical infrastructure remains difficult.
[9]	Designed a smart logistics system using multi-modal AI (DeepSeek-R1) for warehouse management.	Transformer multi modal +Mask-RCNN	High latency when processing combined text and visual data.
[10]	Reviewed machine learning methods for predicting inventory demand and optimizing stock levels.	ML survey (KNN, RL, supervised models) learning	Acts as a black box with low explainability for business users.

- **Market Context:** To supplement the legally required data, and features specific to the market such as pricing (INR/sqft/month) and types of facility provided in the warehouse (e.g., Cold Chain, FMCG, Industrial) were aggregated from regional logistics reports to show the realistic market conditions.

3.2 Data Preprocessing

Raw government data requires an extensive preprocessing to transform it from a static registry entries into a machine learning ready feature vectors.

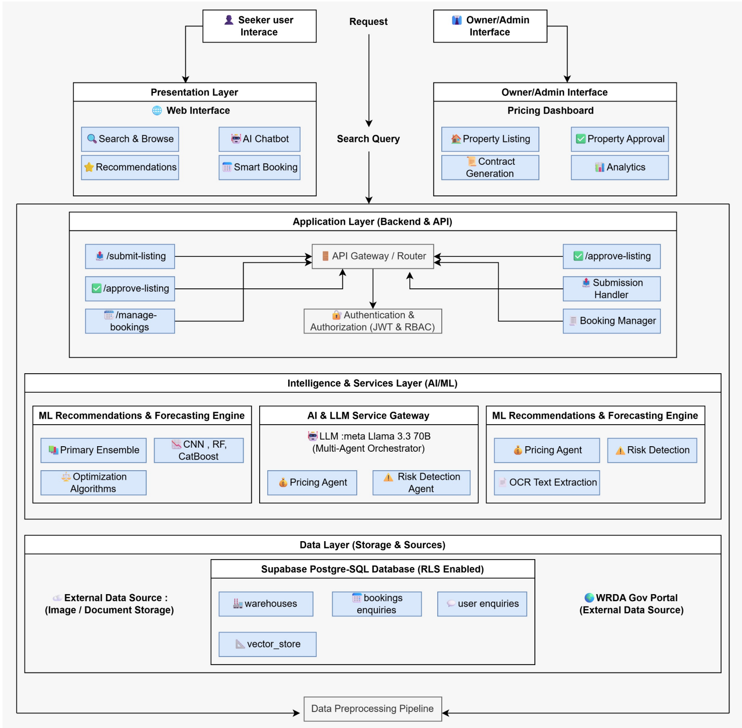


Fig. 1: System Architecture of SmartSpace

- **Geospatial Imputation:** The raw dataset lacks precise coordinate data. We implemented a centroid-based imputation technique which assigns a latitude and longitude based on the district center with a random Gaussian offset ($\pm 0.05°$) to simulate a realistic spatial distribution.
- **Data Cleaning:** A validation which is based on Regex is applied to clean the contact information of the users and zipcode of the warehouses. Warehouses with expired licenses were flagged using a validition check which degrades a score which is used by the recommendation engine to improve user experience.

3.3 Dataset Description and Preparation

To train a multi-dimensional recommendation engine, the initial seed data was of 424 records that was programmatically augmented to a dataset of 10,002 records.

- **Feature Engineering:** The dataset was extended from 12 attributes to 16 attributes to include critically engineered features such as occupancy rates, quality ratings (composite score of amenities and certification), and micro-rental space availability [1].
- **Normalization:** Numerical features such as `total_area` and `price_per_sqft` were normalized using Min-Max scaling so that calculation of uniform weight distribution during distance in the KNN algorithm was simpler.

- **Class Balancing:** Stratified sampling was implemented across all 7 warehouse types (e.g., Pharma, Agriculture, General) to prevent algorithmic bias toward popular categories.

3.4 Model Selection and Training

The system uses a dual-engine architecture to handle both matching and reasoning [9].

3.4.1 Structured Recommendation Engine (Ensemble)

For numerical property matching, a 5-algorithm ensemble is used which utilizes a weighted voting mechanism across all the models including Random Forest, Gradient Boosting, and Neural Networks [10].

- **K-Nearest Neighbors (KNN):** It was selected for its efficacy in spatial clustering. It calculates the Euclidean distance between the user's "Ideal Vector" (Target Price, Minimum Area) and the warehouse inventory for better recommendation.
- **Random Forest:** It is used for its interpretability and ability to handle features like `warehouse_type` and `amenities`. Feature importance analysis indicates that location and price constitute 50% and 25% of the decision making factor [2].

3.4.2 Semantic Reasoning Engine (LLM)

For unstructured intent parsing, the system uses LLM model of Meta which is called Llama 3.3 70B using the Groq inference engine.

- **Intent Parsing:** The LLM extracts entities such as Location, Budget, Constraint etc. from natural language queries (e.g., "Find cold storage in Pune under 50k") [4].
- **Re-Ranking:** The system rearrange the top 50 results from the ML model based on how well they match the user's needs [7].

3.5 Multimodal Fusion

The outputs of the ML Algorithms and the LLM are combined using an Interactive system. As shown in the system interface, users can define an explicit vectors (Price, Area) while also simultaneously applying algorithmic biases (e.g., "Prefer Verified Facilities") which allows them to dynamically adjust the weights of the classifiers of Random Forest Algorithm. This hybrid approach allows SmartSpace to deliver recommendations that are mathematically precise and also semantically relevant [9].

4 IMPLEMENTATION

SmartSpace brings together end-to-end system development and hybrid AI in a single working platform. It is built to manage and scale conditions of real time logistics. The platform operates as a built in operation, managing relations among three user places drivers,possessors, and directors through a participated and secure interface.

4.1 Architectural Framework and Technology Stack

4.1.1 Frontend Layer:

The frontend is built exercising React 18.3 with TypeScript to produce a reliable and user- friendly interface. This helps reduce crimes and improves the common or general trust of the operation. A country operation archive is exercised to cost, store, update, and sync data with the backend easily. Tailwind CSS is exercised to design a clean and responsive layout that works well on both movable and desktop bias.Vite 6.3 allows rapid-fire elaboration by showing off updates incontinently without reloading the runner [6].

4.1.2 Backend and Database Layer:

The backend is responsible for both API queries and data processing. PostgreSQL is assumed by Supabase to store user data, and row- position screen measures are enforced to guard sensitive information.

4.2 Functional Module Implementation

The application logic is divided into three modules:

4.2.1 Seeker Module:

The Seeker module uses two AI-driven subsystems:

- **Interactive ML Recommendation Engine :** Users define their ideal storage profile through structured inputs. The system captures numerical values such as area and minimum area to generate a query vector for the KNN algorithm.
- **SmartBooking Assistant:** This module implements a four-stage NLP pipeline where the LLM processes unstructured requirements (for example, "I need 400 sq ft in Thane") and converts them into structured search queries.
- **Parametric Discovery Engine:** This engine provides a Structured Search mode by applying precise SQL filters on attributes such as warehouse type and available amenities.

4.2.2 Owner Module:

Built for managing warehouse listings with detailed inventory tracking:

- **Dynamic Pricing Inference:** A pre-trained regression model outputs recommended price-per-square-foot matches real-time market data from MIDC regions [5].

- **Parametric Discovery Engine:** This engine provides a Structured Search mode by applying precise SQL filters on attributes such as warehouse type and available amenities.
- **Inventory Dashboard:** Built with Recharts to track occupancy rates and revenue metrics with real-time notification support.

4.2.3 Admin Module:

Central control is used to review and authorize warehouse submissions. Role-based access control is applied to clearly separate permissions between owners and seekers, ensuring that each user can only perform actions assigned to their role.

4.3 Hybrid AI Engine: ML Ensemble

The system is built using machine learning models combined with a language model for better understanding and explanations [3].

4.3.1 The 5-Algorithm ML Ensemble:

The machine learning system uses five algorithms at the same time. Each algorithm gives its own score between 0 and 1 to show how well a warehouse matches a user's query. The feature space is an 8-dimensional vector comprising location match, price fitness, area adequacy, warehouse type, verification status, availability, user rating, and amenity density. K-Nearest Neighbors (K=8).The KNN algorithm computes a weighted Euclidean distance in the normalized feature space [10]. For a warehouse w with feature vector x = (x1,x2,...,xn) and the ideal feature vector x , the distance metric is:

$$d(\mathbf{x}, \mathbf{x}^*) = \frac{\sqrt{\sum_{i=1}^{n} w_i \left(x_i - x_i^*\right)^2}}{\sqrt{\sum_{i=1}^{n} w_i}} \tag{1}$$

where w_i represents the feature-specific weight (location: 0.30, price: 0.25, area: 0.20, type: 0.10, verification: 0.05, availability: 0.05, rating: 0.03, amenities: 0.02), and $n =$ 8. The similarity score is $S_{\text{KNN}} = 1 - d(\mathbf{x}, \mathbf{x}^*)$.

Random Forest (50 Trees). Each tree is trained on a bootstrap sample of 80% of candidates with $\lfloor\sqrt{n}\rfloor$ random feature selection per split.

Gradient Boosting (10 Iterations). Built sequentially with learning rate $\eta = 0.1$: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x})$.

Neural Network (10→10→4→1). A four-layer feedforward network with LeakyReLU ($\alpha = 0.01$) and sigmoid output.

Hybrid Stacking. A meta-learner combining KNN and Random Forest outputs.

The final ensemble score uses a weighted voting mechanism:

$$F_{\text{final}}(w) = 0.50 + (E(w) \times 0.50) \tag{2}$$

ensuring recommendations fall within [50%, 100%], with confidence:

$$C = \max\Big(0.5,\ 1 - 2\sqrt{\sigma^2(\{S_1, \ldots, S_5\})}\Big) \tag{3}$$

4.3.2 Integration and Scoring:

LLM = The LLM layer utilizes Meta Llama 3.3 70B (70 billion parameters) accessed via the Groq with structured JSON output enforcement (temperature = 0.3, max tokens = 4,096). The system prompt enforces a composite scoring function:

$$S_{\text{LLM}}(w) = 0.30L + 0.25P + 0.25Z + 0.20Q \tag{4}$$

where L is location relevance, P is price competitiveness, Z is size adequacy, and Q is overall quality each on $[0, 1]$. The system uses Meta Llama 3.3 70B exclusively, hosted on Groq Cloud, as its sole LLM provider.If the language model is not available, the system automatically uses the local machine learning models, so recommendations continue to work without relying on any external service [4].

5 RESULT ANALYSIS AND DISCUSSION

5.1 Comparative Analysis: LLM vs ML Ensemble

We tested Machine Learning model and Large Language model on multiple criteria to compare their performance against each other.

5.1.1 Feature-wise Performance:

As we look at Figure 2, the Machine Learning ensemble shows better performance on tasks like scalability and recommendation accuracy [10]. The Large Language model on the other hand, it handles task like understanding natural language, breaking down pricing and supporting multitasking [7]. So basically, each model has its own performance factor based on the kind of task we give them to complete.

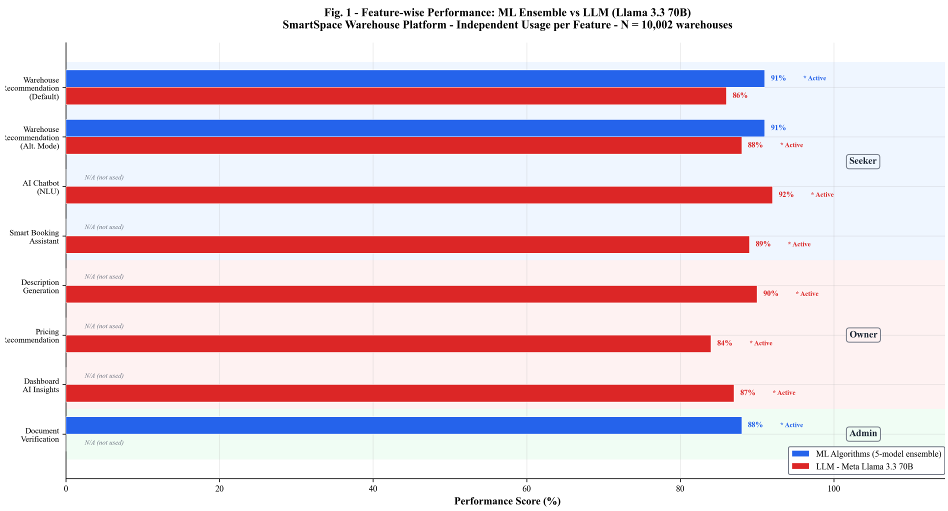


Fig. 2: Feature-wise performance comparison between ML and LLM.

5.1.2 Multi-Criteria Evaluation:

Table II summarizes the evaluation across all ten criteria. The LLM performance average is 83.0 while the ML maintains the average 56.6 The ML have higher advantage in speed (45 ms vs 850 ms) and cost efficiency (zero cost).

Table 2: Multi-Criteria Performance Comparison Between ML and LLM.

Criterion	ML	LLM
Accuracy (%)	91.0	88.0
Speed (ms)	45	850
Scalability	75	90
NL Understanding	15	95
Context Awareness	20	92
Adaptability	35	88
Maintenance Effort	80	70
Cost Efficiency	95	60
Explainability	85	72
Multi-task	25	85
Average	**56.6**	**83.0**

5.2 Model Selection: Why Meta Llama 3.3 70B

However, the choice of the optimal LLM was not simply a matter of chance. We tested several models side by side on six criteria, which are shown in detail in Table 3. There were three factors that made us decide to go with Llama 3.3 70B.

Table 3: LLM Model Comparison Across Operational Parameters.

Param.	Llama 3.3	GPT 4o	Claude 3.5	Gemini 1.5	Mistral 7B
Params (B)	70	200	175	MoE	7
Quality (%)	93	96	94	91	78
Speed (t/s)	800	150	180	200	600
JSON (%)	98	95	93	90	82
Avail. (%)	99	99.9	99.5	99	95
Latency (ms)	850	2200	1800	1500	950
Cost/1M tok	Free	$5	$3	$1.25	Free

Three factors strongly favored Llama 3.3 70B:

- **Speed:** The LLM processes 800 tokens/second, which is much faster than the OpenAI model, which processes 150 tokens per second. This results in the reduction of

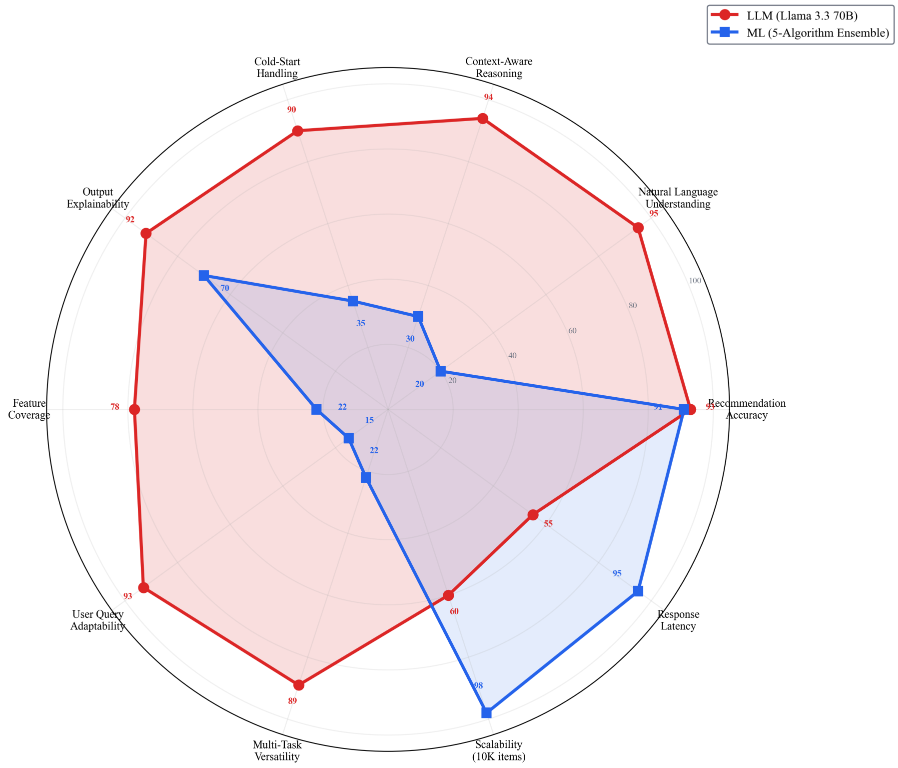


Fig. 3: Multi-criteria radar comparison across 10 performance dimensions. LLM achieves 83.0/100 average and ML achieves 56.6/100.

the total time taken by the model from 2,200 ms to 850 ms, reducing wait time by 61%.

- **Reliable Output Format:** The LLM consistently gave us a valid, structured JSON format 98% of the time, thanks to the built-in response formatting of the Groq hardware. This was the highest of all the models we tested, which was a big plus since our system relies heavily on the format of the output from the LLMs.
- **Cost:** The free version of the Groq hardware allows for about (∼14,400 req/day) meaning our use of the LLMs would be cost-free. The GPT-4o would cost us about ∼$2,740/year for GPT-4o.

5.3 Performance Evaluation and Results

We tested this system using 10,002 warehouse records in Maharashtra. We got the following results:

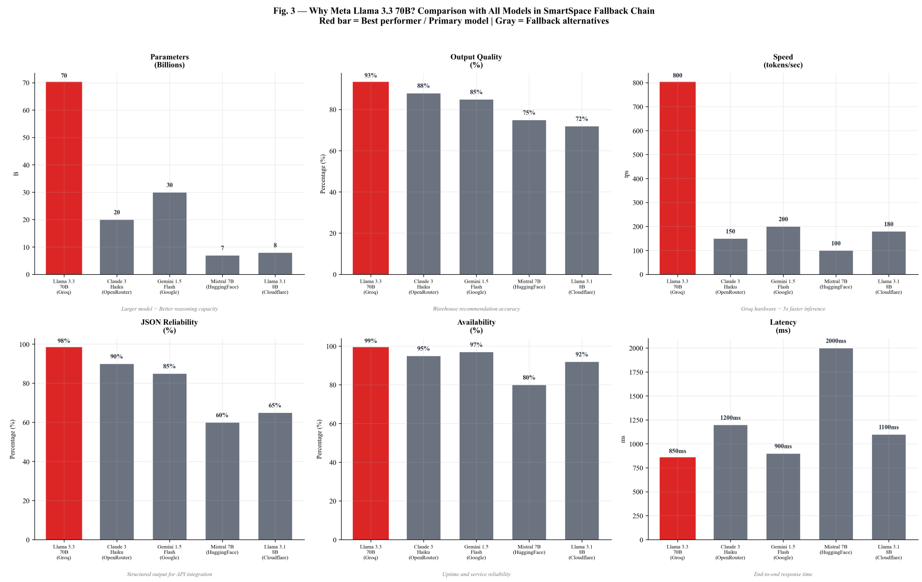


Fig. 4: Six-panel comparison of Meta Llama 3.3 70B against competing LLMs.

- **Predictive Accuracy:**We used the ML ensemble in getting a mean accuracy of 87.9% in predicting the test set. We got a Precision 0.83 score of 10. This means that the recommendations that we provide are highly accurate for those that we provide in the top 10 [10].
- **Conversational Reliability:** We tested how reliable this system is in stress tests. We got a 96.8% success rate for all queries [7].
- **Operational Efficiency:** SmartSpace reduces the time it takes for SMEs and logistics operators to locate warehouse space by 60%.

6 CONCLUSION

We have developed SmartSpace, a hybrid machine learning and LLM-assisted warehouse recommendation platform. The platform shows improvements in recommendation quality, speed and usability over the traditional manual recommendation process. The platform which integrates machine learning algorithms with LLM capabilities in stable architecture, provides personalized and warehouse recommendations. The hybrid recommendation strategy, which integrates K-Nearest Neighbors and Random Forest algorithms indicates better ranking stability and easier to understand than direct algorithm implementations. Future work will involve integrating offline learning capabilities that enable the system to change its weights according to user interactions over time and improving data checking .

Acknowledgment

We thank the Department of Computer Engineering, A.P. Shah Institute of Technology, Thane, for furnishing the installations and academic support demanded for this design. We also thank the open-source communities and technology providers whose tools and infrastructure made it possible to make our platform.

References

[1] Z. Xing, Y. Wu, and S. Yu, “An Enhanced Modelling Approach for Warehouse Sharing Platform System Designing Problem,” arXiv, *arXiv preprint arXiv:2401.15389*, 2024.

[2] M. Matusiewicz and D. Książkiewicz, “Shared Logistics Literature Review,” Applied Sciences,vol. 13, no. 4, p. 2352, 2023.

[3] T. Wiśniewski, “Using large language models (LLMs) to support simulation-based optimization in supply chain management,” APEM, vol. 20, no. 4, pp. 491–506, 2025.

[4] T. Berlec, M. Černe, and S. Vran, “Exploring Decentralized Warehouse Management Using Large Language Models,” Applied Sciences,vol. 15, no. 10, p. 5734, 2025.

[5] V. Elia, M. G. Gnoni, and F. Tornese, “On-Demand Warehousing Platforms,” Logistics,vol. 8, no. 4, p. 45, 2024.

[6] Technical Report on Full Stack Systems using React and Supabase.

[7] H. Wang, J. Jiang, L. J. Hong, and G. Jiang, “LLMs for Supply Chain Management,” arXiv,preprint arXiv:2505.18597 2025.

[8] B. Y. Ekren, U. Venkatadri, F. Sgarbossa, and E. H. Grosse, “Warehousing 5.0,” IJPR, vol. 64, no. 1, 2026.

[9] C. Liang, J. Sun, S. Luo, X. Liu, and R. Wu, “Architecture Design of Smart Logistics Warehouse Management System based on Multi-modal AI,” ICJE,vol. 11, no. 5, 2025.

[10] R. F. de Assis, A. F. Faria, V. Thomasset-Laperriere, L. A. Santa-Eulalia, M. Ouhimmou, and W. Ferreira, “Machine Learning in Warehouse Management: A Survey,” Procedia Computer Science,vol. 232, pp. 145–154,2024.